OWASP API Top 10

Security Assessment Lab

Practical Exploitation & Remediation Guide

Table of Contents

1. Lab Environment & Tools 2
crAPI, Docker Desktop, Burp Suite 2

2. Lab Setup & Configuration 2
Prerequisites 2
Clone Repository 2
Environment Configuration 2
Start crAPI Services 2
Access Points 2
Initial Setup Verification 3

3. OWASP Overview 3
OWASP API Security Top 10 (2023) 3

4. Vulnerability Exploitation 4
4.1 API1:2023 — Broken Object Level Authorization (IDOR) 4
4.2 API2:2023 — Broken Authentication 5
4.3 API3:2023 — Broken Object Property Level Authorization 7
4.4 API4:2023 — Unrestricted Resource Consumption 8
4.5 API5:2023 — Broken Function Level Authorization 9
4.6 API6:2023 — Unrestricted Access to Sensitive Business Flows 11
4.7 API7:2023 — Server-Side Request Forgery (SSRF) 12
4.8 API8:2023 — Security Misconfiguration 13
4.9 API9:2023 — Improper Inventory Management 14

5. Summary of Findings 15

1. Lab Environment & Tools

This lab uses three core components to simulate real-world API security testing. Each tool plays a distinct role in the exploitation workflow.

Tool	Role	Description
crAPI	Target Application	Completely Ridiculous API (crAPI) — a deliberately vulnerable web application by OWASP, providing a legal and hands-on training ground to practice identifying and exploiting modern API vulnerabilities.
Docker Desktop	Container Runtime	A cross-platform GUI application for building, running, and managing containerized applications. Simplifies lab environment setup by packaging all services into isolated containers.
Burp Suite	Interception Proxy	An industry-leading penetration testing platform that intercepts HTTP/HTTPS traffic between the browser and target server, enabling real-time capture, inspection, and modification of API requests.

2. Lab Setup & Configuration

2.1 Prerequisites

Ensure the following software is installed before proceeding:

- Docker (latest stable version)
- Docker Compose
- Git
- VS Code or any IDE (optional, for source exploration)

Verify installations using the following commands:

```
docker --version
docker-compose --version
git --version
```

2.2 Clone the crAPI Repository

Download the official crAPI project from the OWASP GitHub repository:

```
git clone https://github.com/OWASP/crAPI.git
cd crAPI
```

2.3 Environment Configuration

> **Note**
> The docker-compose.yml file inside the cloned folder defines all services (API, frontend, database, mail server, etc.). No changes are required for the default setup. Ensure ports 8888 and 8025 are not in use before starting.

2.4 Start crAPI Services

Launch all containers in detached mode:

```
docker-compose up -d
```

Verify that all containers are running:

```
docker ps
```

2.5 Access Points

Service	URL
Frontend (Web App)	http://localhost:8888
MailHog (Email Testing)	http://localhost:8025

2.6 Initial Setup Verification

Complete the following steps to confirm the lab environment is operational:

- Open the frontend in your browser at http://localhost:8888
- Register a new user account
- Confirm the registration email via the MailHog interface
- Log in to verify the application is fully functional

To shut down the lab environment when finished:

```
docker-compose down
```

3. OWASP Overview

The Open Worldwide Application Security Project (OWASP) is a globally recognized non-profit organization dedicated to improving the security of software. It provides freely accessible resources, tools, and guidelines to help developers, testers, and organizations build and maintain secure applications.

OWASP is the industry standard for API security awareness. Its Top 10 lists represent the most critical security risks identified through community research and real-world data and are widely adopted across organizations worldwide to guide secure development practices and penetration testing frameworks.

3.1 OWASP API Security Top 10 (2023)

#	Vulnerability ID	Name
1	API1:2023	Broken Object Level Authorization
2	API2:2023	Broken Authentication
3	API3:2023	Broken Object Property Level Authorization
4	API4:2023	Unrestricted Resource Consumption
5	API5:2023	Broken Function Level Authorization
6	API6:2023	Unrestricted Access to Sensitive Business Flows
7	API7:2023	Server-Side Request Forgery (SSRF)
8	API8:2023	Security Misconfiguration
9	API9:2023	Improper Inventory Management
10	API10:2023	Unsafe Consumption of APIs

4. Vulnerability Exploitation

The following sections present a comprehensive analysis of the hands-on exploitation of vulnerabilities from the OWASP API Security Top 10 using the crAPI (Completely Ridiculous API) laboratory environment. Each vulnerability is introduced with a technical explanation that outlines its underlying security concepts, common causes, and potential impact on the confidentiality, integrity, and availability of application data and services. This theoretical foundation provides the context necessary to understand how insecure API implementations can be abused by malicious actors and why these vulnerabilities remain prevalent in modern web applications and microservice architectures.

Following the technical overview, each section details the practical exploitation process carried out within the crAPI environment. The methodology includes reconnaissance activities, endpoint identification, request manipulation, parameter analysis, authentication and authorization testing, and the use of common security testing tools such as Burp Suite and API clients. Relevant API requests and responses are examined to demonstrate how vulnerability can be identified, validated, and exploited in a controlled setting. To support the findings, screenshot evidence is provided throughout the assessment, illustrating key stages of the exploitation process and confirming successful execution of each attack. Together, these demonstrations highlight the real-world risks associated with insecure APIs and emphasize the importance of implementing effective security controls to mitigate such vulnerabilities.

API1

API1:2023

Broken Object Level Authorization (IDOR)

Overview

Broken Object Level Authorization (BOLA) occurs when an application properly authenticates a user but fails to verify whether that user has permission to access a specific resource. This vulnerability arises when the server trusts object identifiers, such as user IDs, account numbers, or record IDs, provided by the client without performing adequate authorization checks. As a result, an attacker can manipulate these identifiers to access, modify, or delete resources belonging to other users, leading to unauthorized data exposure and potential compromise of sensitive information.

> **Definition: IDOR**
> Insecure Direct Object Reference (IDOR) is a specific manifestation of BOLA where an application exposes direct references to internal database objects (e.g., user IDs, vehicle IDs) and fails to validate access permissions for the requesting user.

Exploitation Steps

- Register and log in to the crAPI application. Explore the dashboard, community tab, and shop features.
- Navigate to the dashboard and click the Refresh button. Capture the resulting HTTP request in Burp Suite.
- Inspect the captured request to identify the vehicle ID parameter associated with your account.
- Browse the community tab and identify other users' vehicle IDs exposed in the API response (e.g., user jackripper).
- In Burp Suite Repeater, substitute your vehicle ID with the victim's vehicle ID in the location request.

Evidence

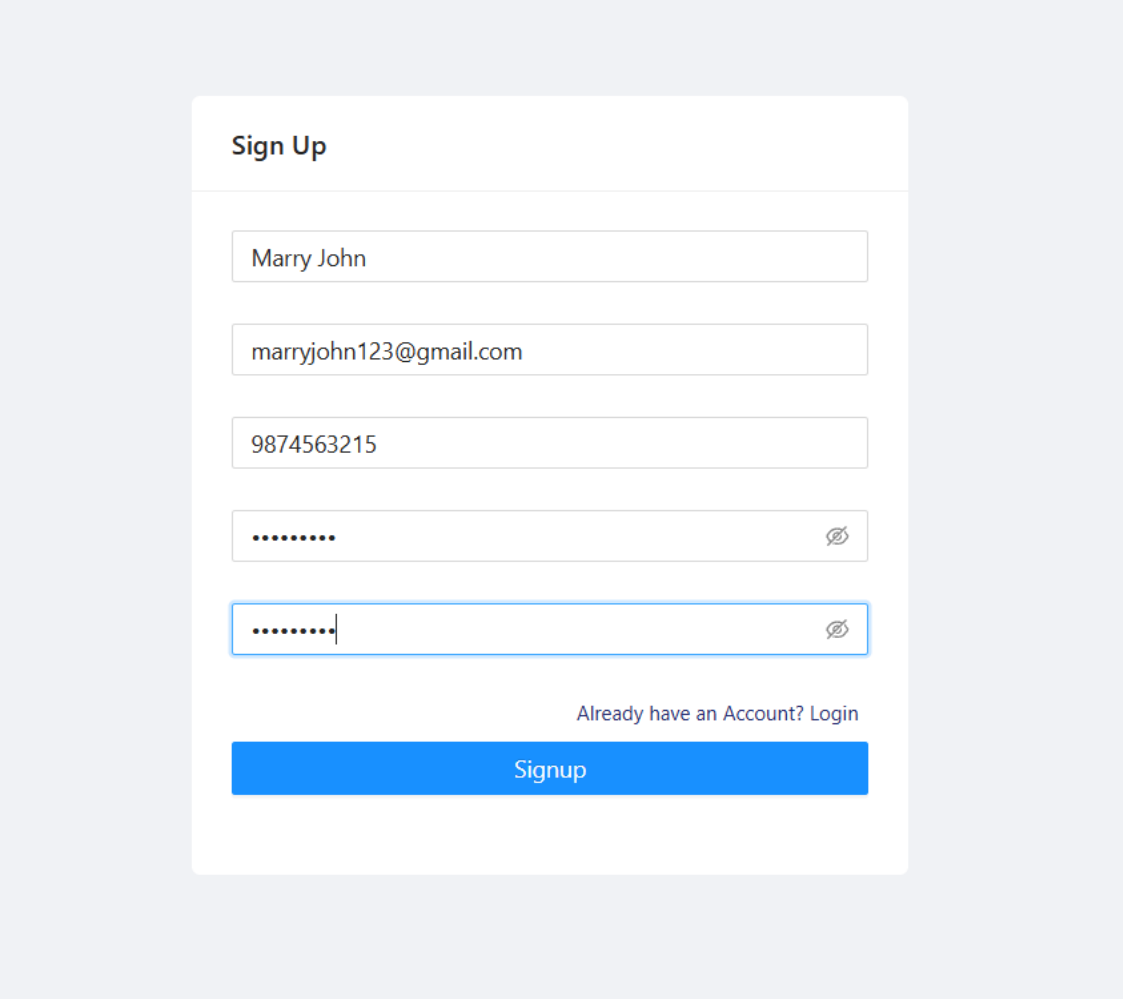

Figure 1.1 — Signup the page

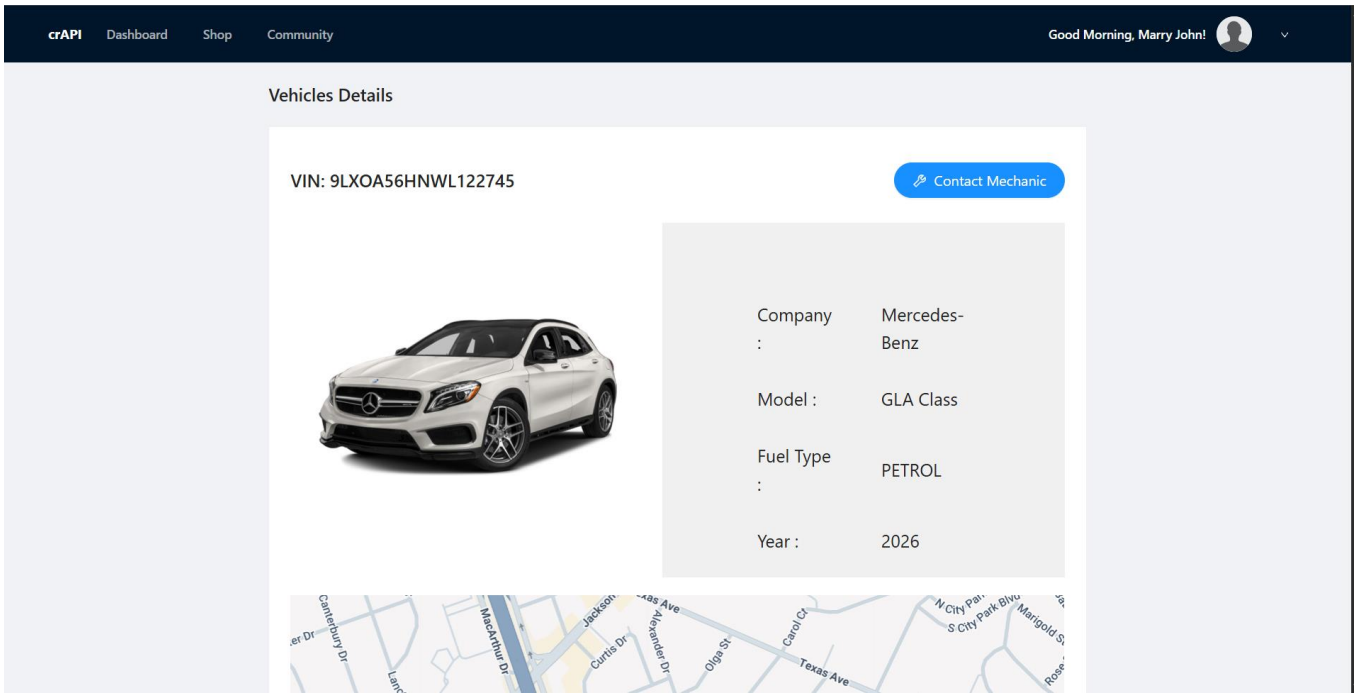

Figure 1.2 — crAPI home page after login showing vehicle dashboard

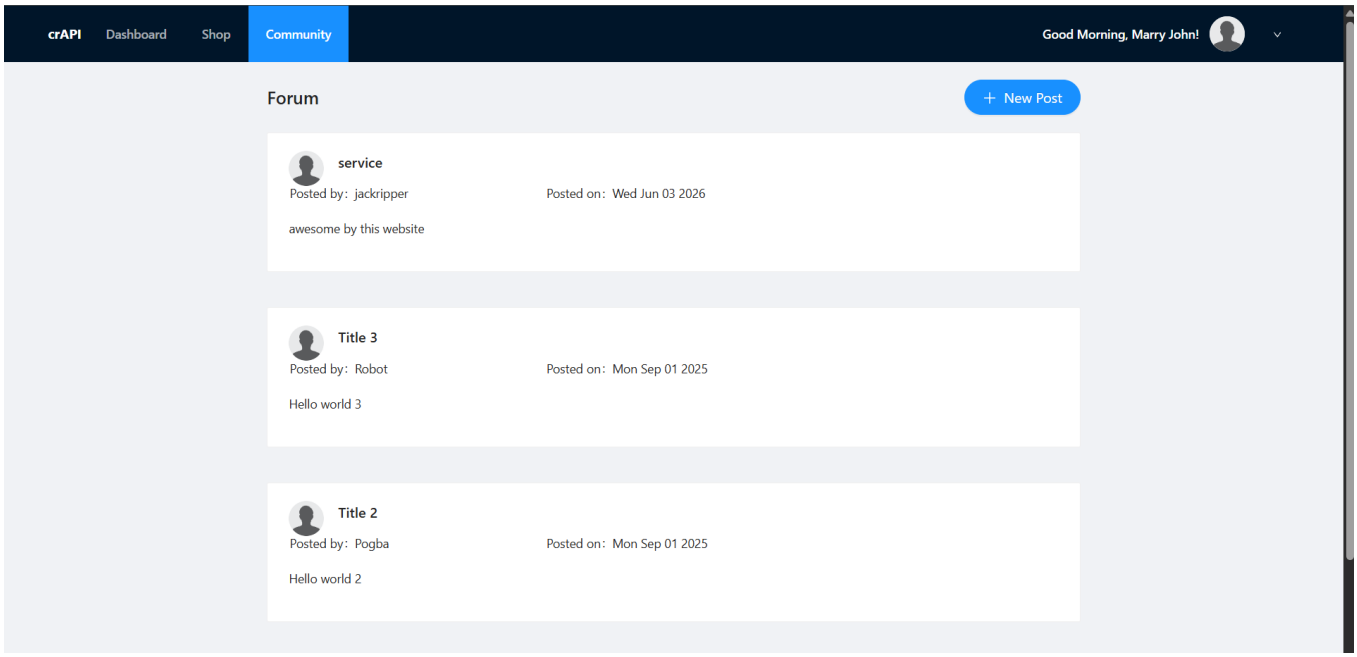

Figure 1.3 — Exploring application features

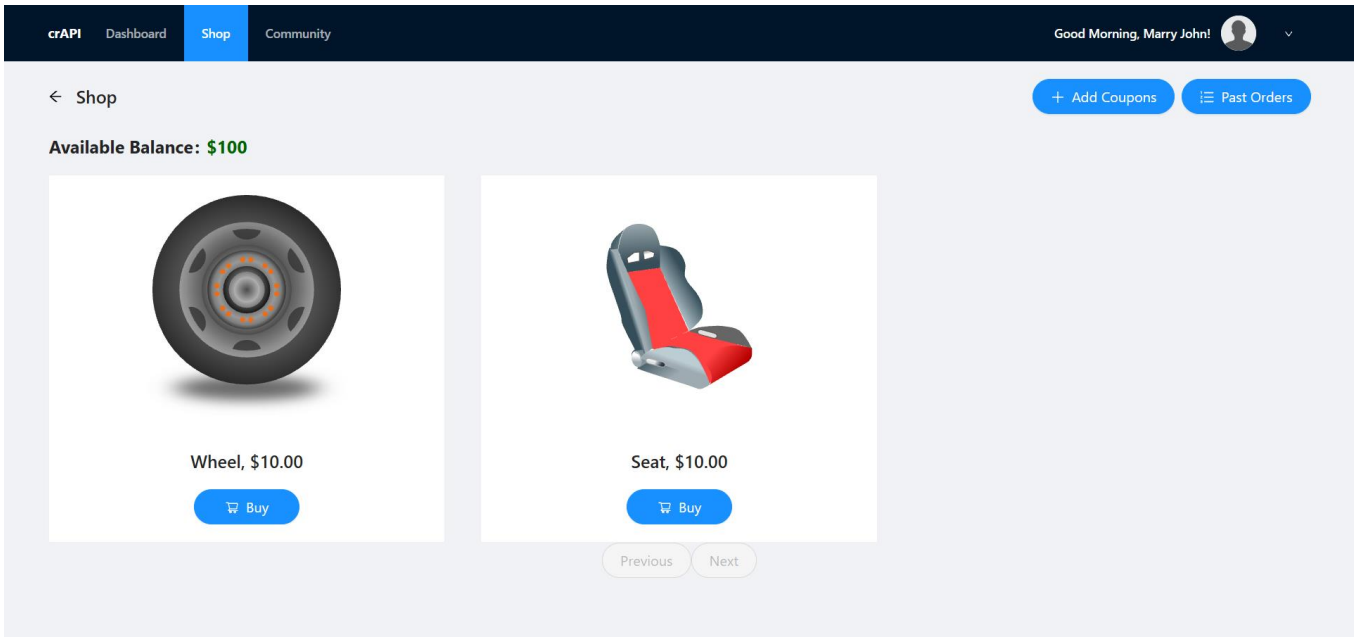

Figure 1.4 — Exploring application features

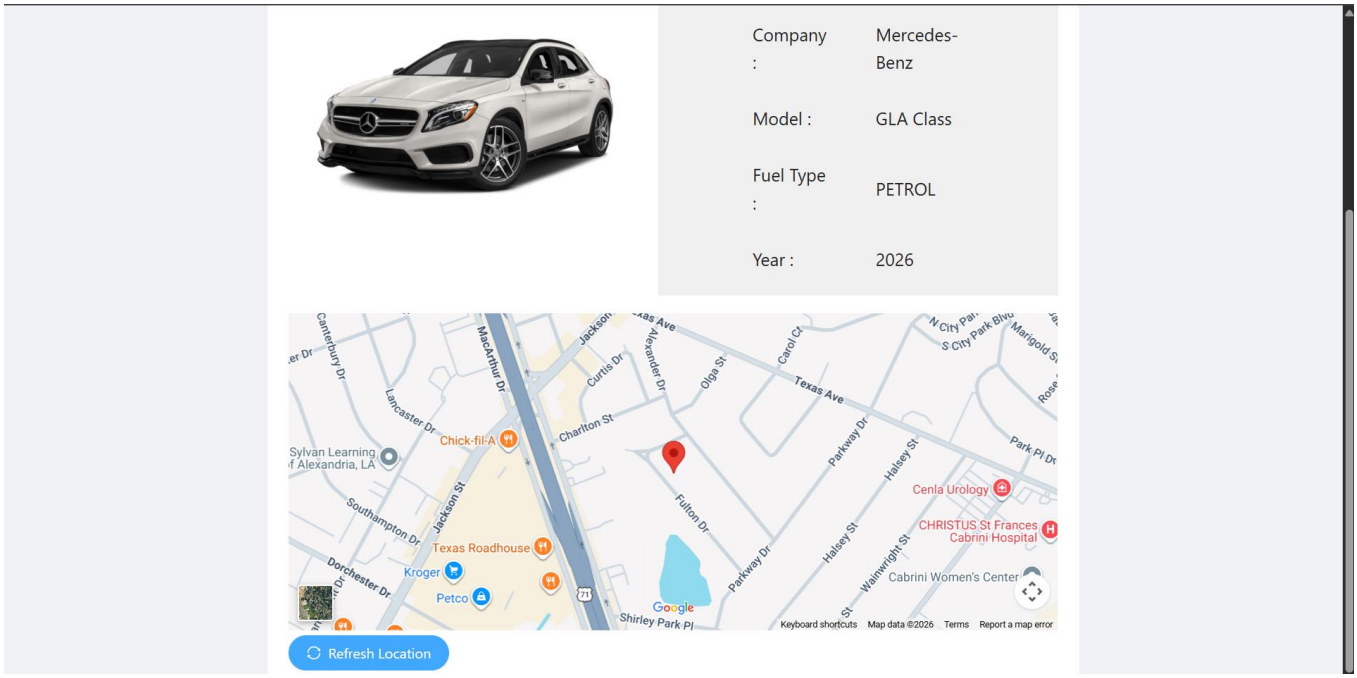

Figure 1.5 — Dashboard with Refresh button to trigger location request

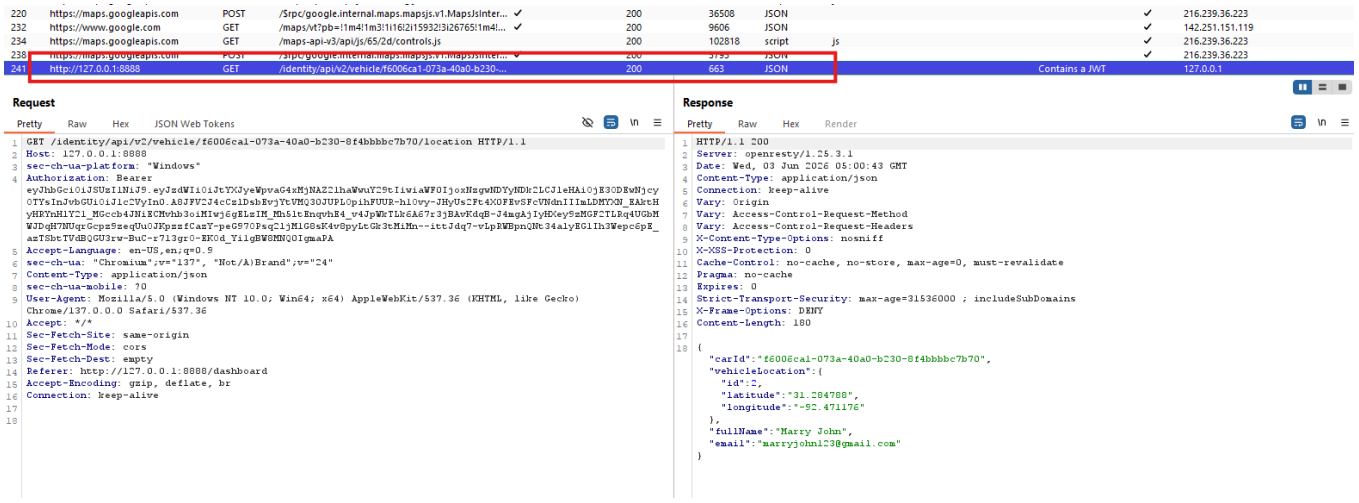

Figure 1.6 — Burp Suite capturing the vehicle location request & Vehicle ID and user details visible in captured request

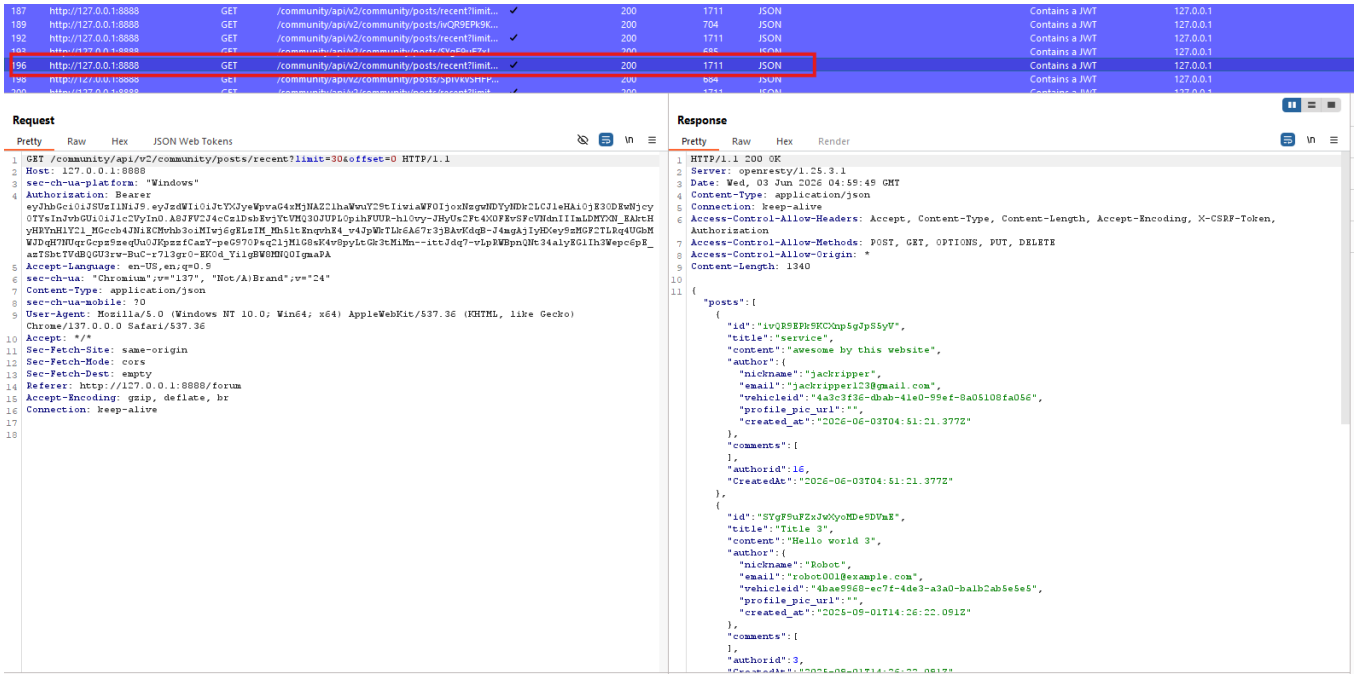

Figure 1.7 — Community tab revealing other users' vehicle IDs in API response

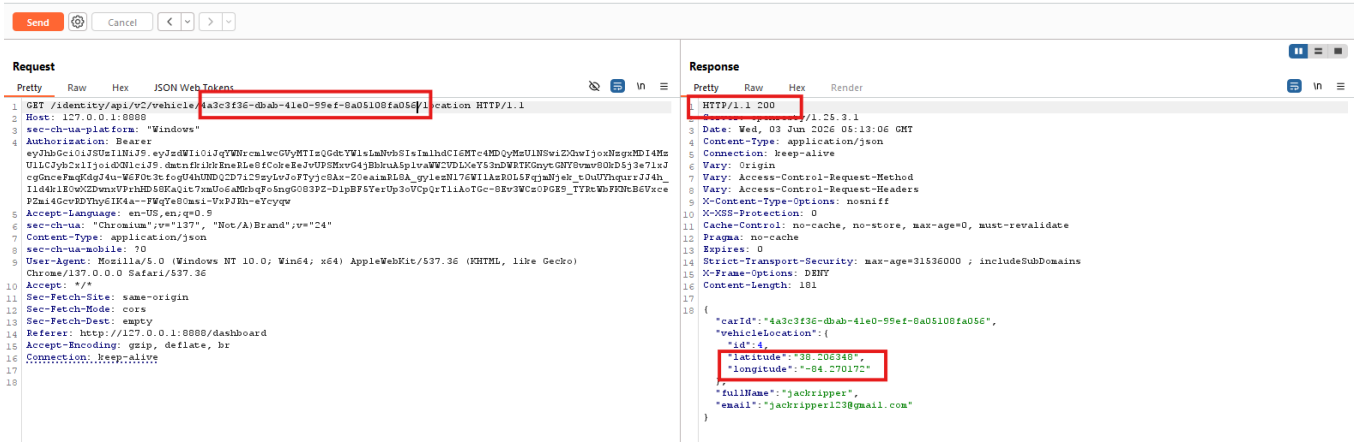

Figure 1.8 — Substituting victim vehicle ID confirms unauthorized location access (IDOR confirmed)

Confirmed Vulnerability

By substituting the authenticated user's vehicle ID with that of user "jackripper", the API returned to the target user's real-time vehicle location. This data should be accessible only to the owner, confirming an IDOR / BOLA vulnerability.

Remediation

- Implement **server-side authorization checks** for every request that accesses, modifies, or deletes a resource.
- Verify that the **authenticated user owns the requested object** or has the necessary permissions before granting access.
- Avoid relying on **client-supplied object identifiers** (e.g., user IDs, account IDs) without validating ownership.
- Enforce the **principle of least privilege**, ensuring users can only access resources required for their role.
- Centralize access control logic to ensure **consistent authorization of enforcement** across all API endpoints.
- Use **role-based access control (RBAC)** or attribute-based access control (ABAC) mechanisms where appropriate.
- Consider using **unpredictable identifiers** (e.g., UUIDs) to reduce the risk of object enumeration, while still enforcing authorization checks.
- Log and monitor unauthorized access attempts to detect potential attacks and suspicious activity.
- Conduct regular **security assessments, code reviews, and penetration testing** to identify authorization weaknesses.
- Implement automated security testing in the development lifecycle to prevent authorization flaws from reaching production.

API2

API2:2023

Broken Authentication

Overview

Broken Authentication is a vulnerability that occurs when an application improperly implements authentication, credential management, or session handling mechanisms, allowing attackers to compromise user identities and gain unauthorized access to accounts. Authentication systems are responsible for verifying that users are who they claim to be; however, weaknesses in their design or implementation can undermine this process and enable account takeover attacks.

Unlike vulnerabilities that exploit coding errors or software flaws, Broken Authentication often results from weak security controls and poor architectural decisions. Common issues include allowing weak or easily guessable passwords, failing to enforce password complexity requirements, permitting the reuse of compromised credentials, and not implementing multi-factor authentication (MFA) for sensitive accounts. Attackers can exploit these weaknesses through techniques such as brute-force attacks, password spraying, credential stuffing, and social engineering.

Session management weaknesses can further increase the risk. For example, predictable or insecure session tokens, failure to invalidate sessions after logging out, excessively long session lifetimes, or improper handling of authentication cookies can allow attackers to hijack active user sessions. Additionally, inadequate protection of login and verification endpoints—such as failing to rate-limit login attempts or One-Time Password (OTP) verification requests—can enable automated attacks that systematically guess credentials or authentication codes.

Successful exploitation of Broken Authentication can lead to account compromise, unauthorized access to sensitive data, privilege escalation, fraudulent transactions, and complete takeover of user or administrative accounts. Because authentication serves as the foundation of application security, weaknesses in this area can have severe consequences for confidentiality, integrity, and availability of system resources.

Exploitation Steps

- Using the email address harvested during the BOLA exploitation (jackripper123@gmail.com), navigate to the Forgot Password page.
- Submit the OTP reset request. Since direct mailbox access is unavailable, capture the OTP verification request in Burp Suite.
- Forward the captured request to the Burp Suite Intruder tab. Configure a Sniper attack with a numeric payload ranging from 0000 to 9999 to brute-force the 4-digit OTP.
- Execute the attack. Monitor responses for an HTTP 200 status code, indicating a successful OTP match.

Evidence

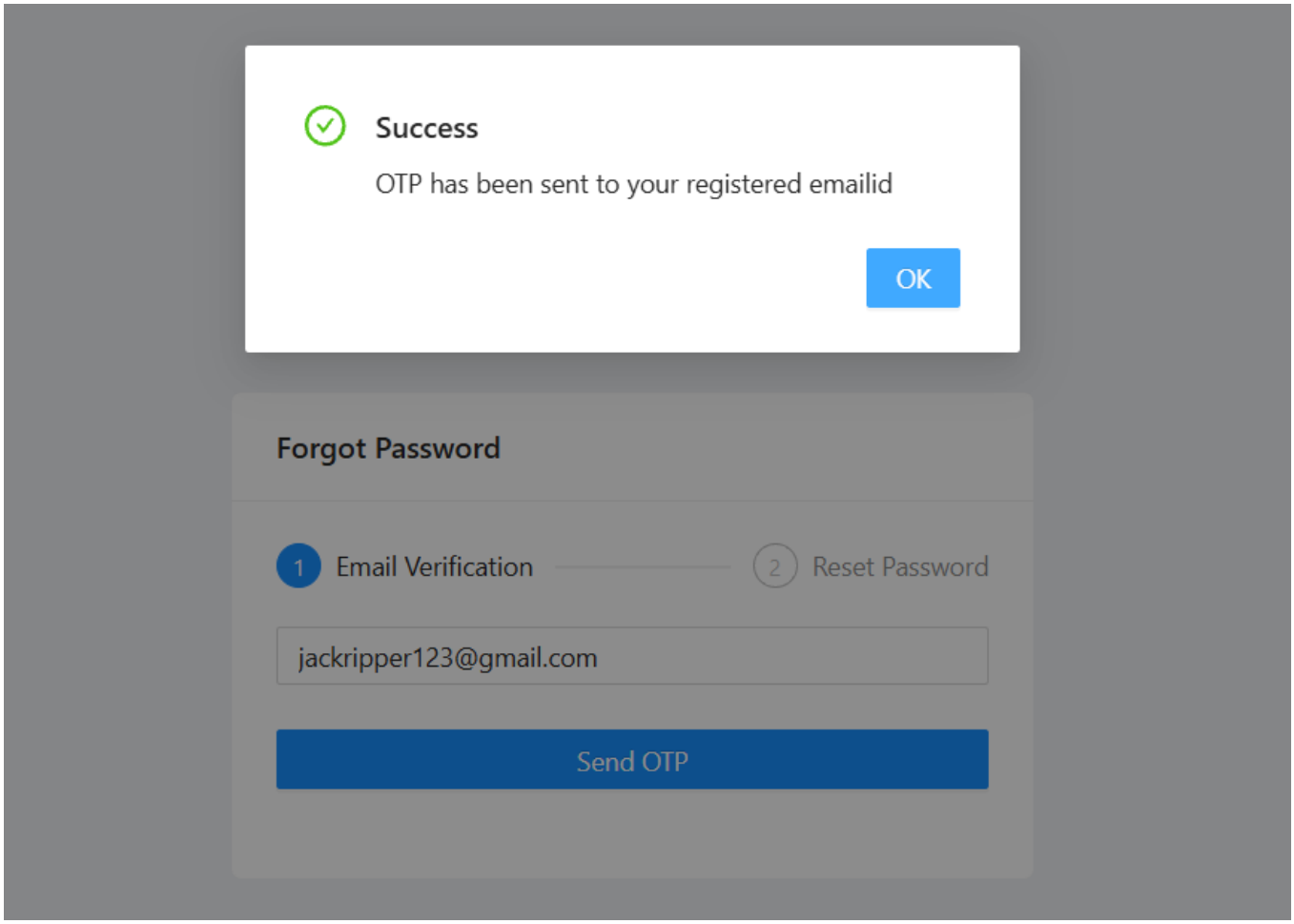

Figure 2.1 — Forgot Password page with victim's email address entered

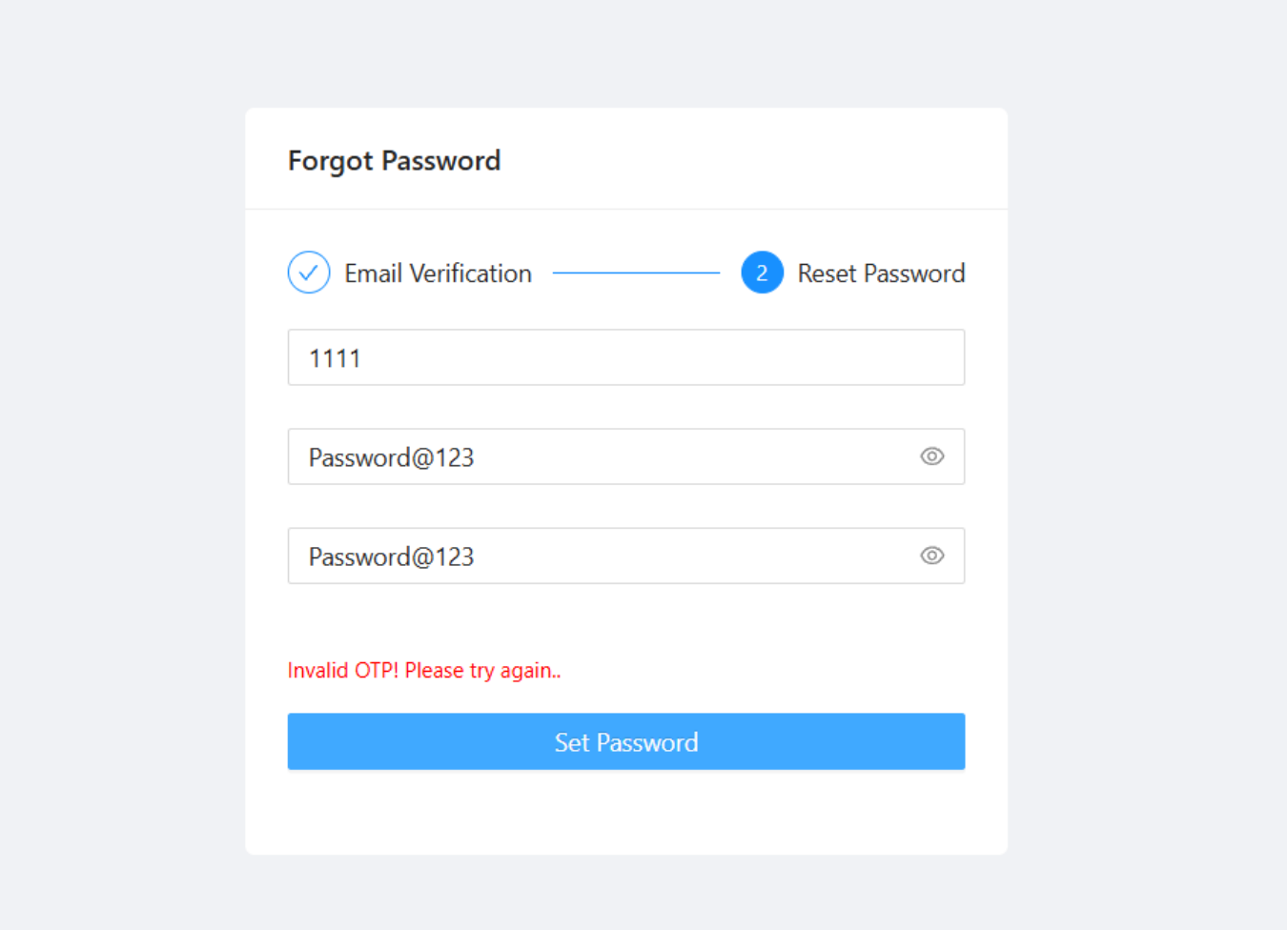

Figure 2.2 — OTP sent to mail; attempting reset with random OTP

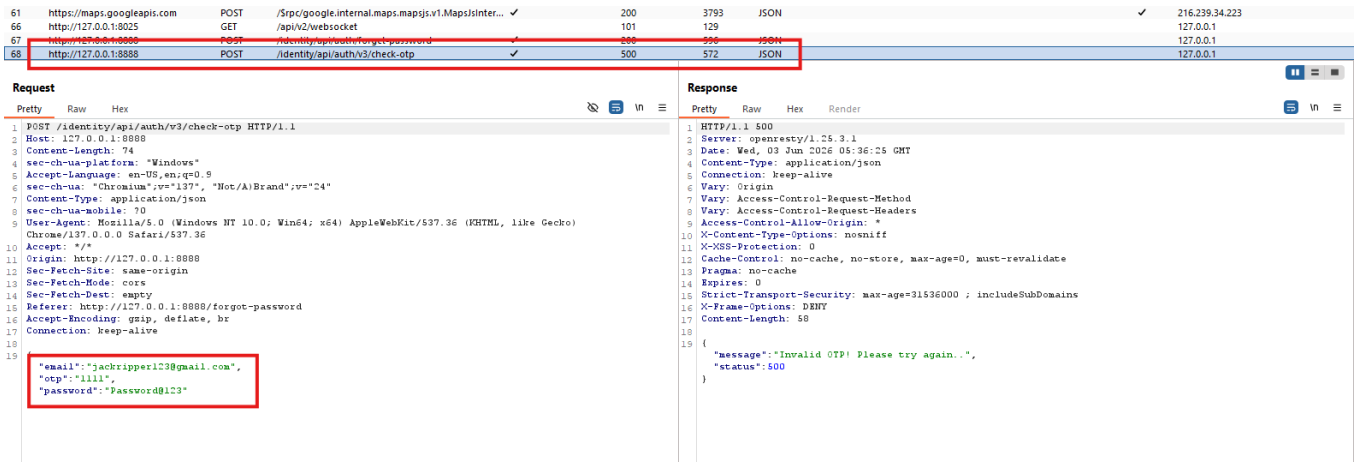

Figure 2.3 — Invalid OTP error captured in Burp Suite

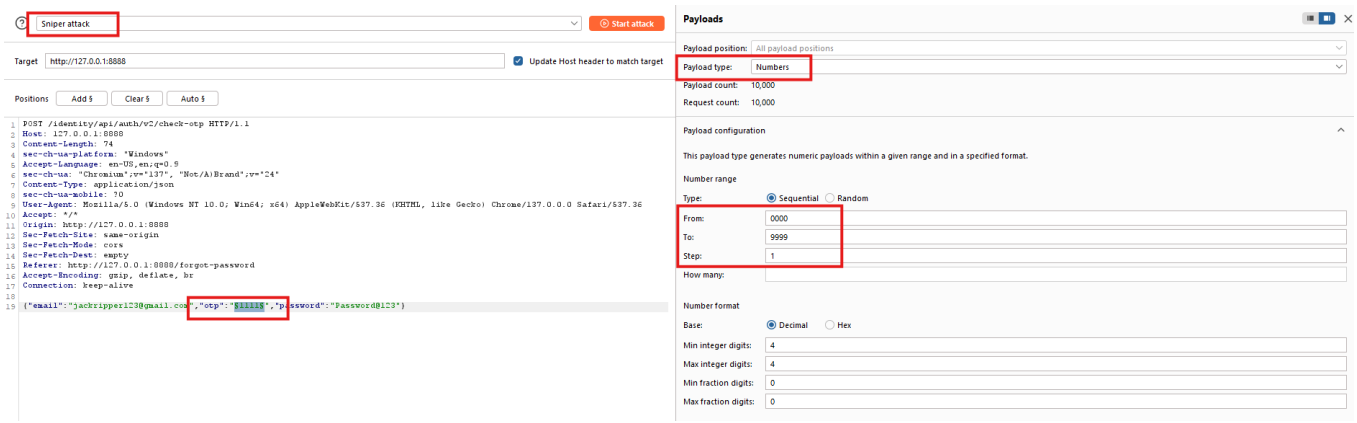

Figure 2.4 — Intruder configured with numeric payload (0000–9999) for Sniper attack, payload range configuration

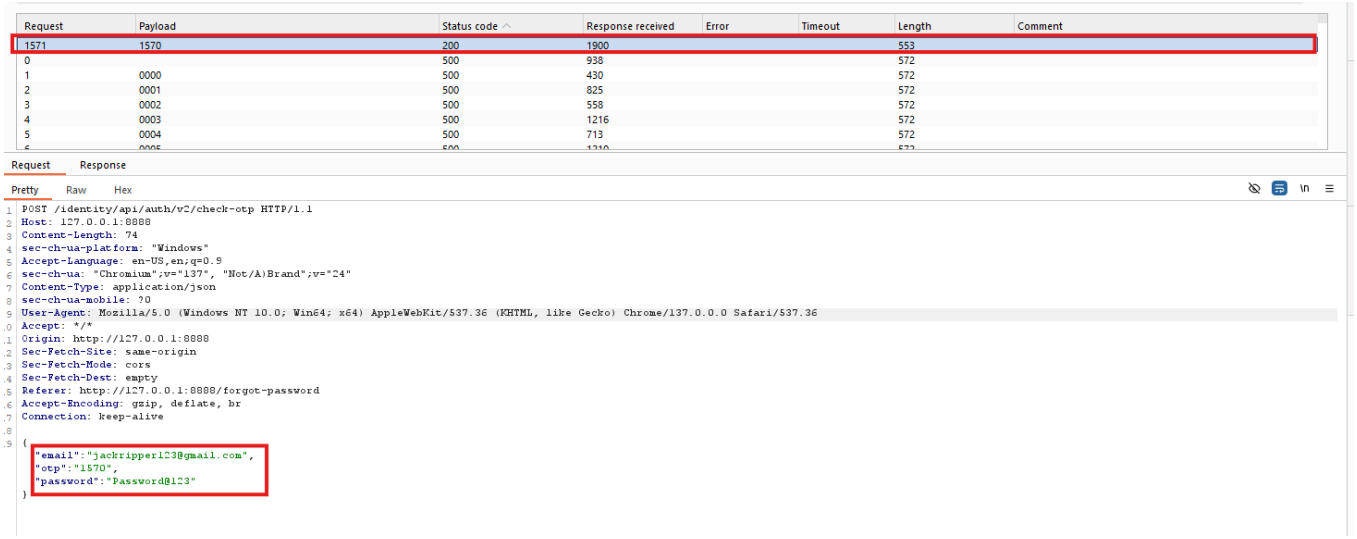

Figure 2.5 — HTTP 200 response confirms matching OTP

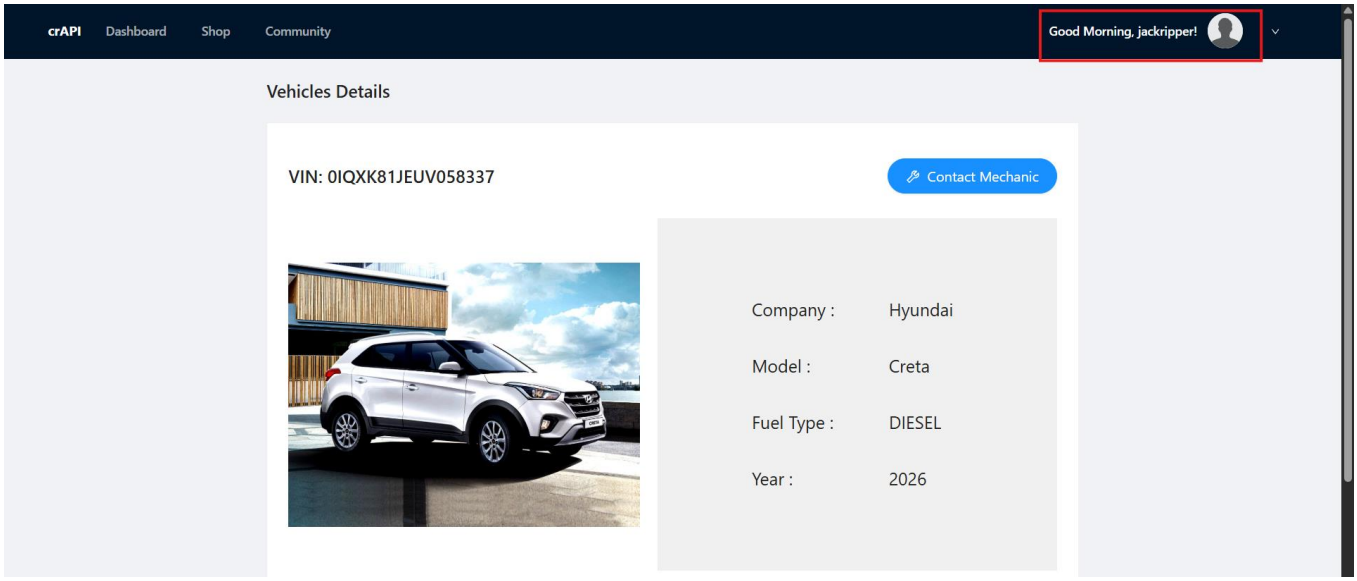

Figure 2.6 — Account password reset and login successful

Confirmed Vulnerability

The attack successfully identified a matching OTP and reset the password for the jackripper account. Subsequent login with the new credentials confirmed full account takeover, demonstrating a complete Broken Authentication vulnerability due to absent OTP rate-limiting.

Remediation

- Enforce strong password policies, including minimum length, complexity requirements, and prevention of commonly used or compromised passwords.
- Implement Multi-Factor Authentication (MFA) to provide an additional layer of security beyond passwords.
- Apply rate limiting and account lockout mechanisms to prevent brute-force, credential stuffing, and password-spraying attacks.
- Use secure password hashing algorithms such as bcrypt, Argon2, or PBKDF2 to protect stored credentials.
- Generate secure session tokens and invalidate them upon logout, password changes, or prolonged inactivity to prevent session hijacking.

API3

API3:2023

Broken Object Property Level Authorization

Overview

Broken Object Property Level Authorization (BOPLA) is a vulnerability that occurs when an API correctly verifies that a user is authorized to access a particular resource but fails to enforce proper authorization controls on the individual properties or fields within that resource. As a result, users may be able to view or modify sensitive data elements that should be restricted, even though they are permitted to access the resource itself.

This vulnerability typically manifests in two forms: **Excessive Data Exposure** and **Mass Assignment**. Excessive Data Exposure occurs when an API response includes sensitive fields that are not required by the client, such as internal identifiers, account statuses, administrative flags, personal information, or other confidential attributes. Although the user may have legitimate access to the resource, they should not be able to view these specific properties. Mass Assignment occurs when an API automatically maps user-supplied input to backend object properties without validating which fields are allowed to be modified. Attackers can exploit this behavior by adding unauthorized parameters to a request and altering sensitive attributes, such as account roles, permissions, account balances, or administrative settings.

Exploitation Steps

- Log in and navigate to the community tab to view user posts.
- Capture a community feed request in Burp Suite. Unlike the browser, Burp Suite does not apply JavaScript-level response filtering.
- Inspect the raw API response to enumerate all returned object properties.

Evidence

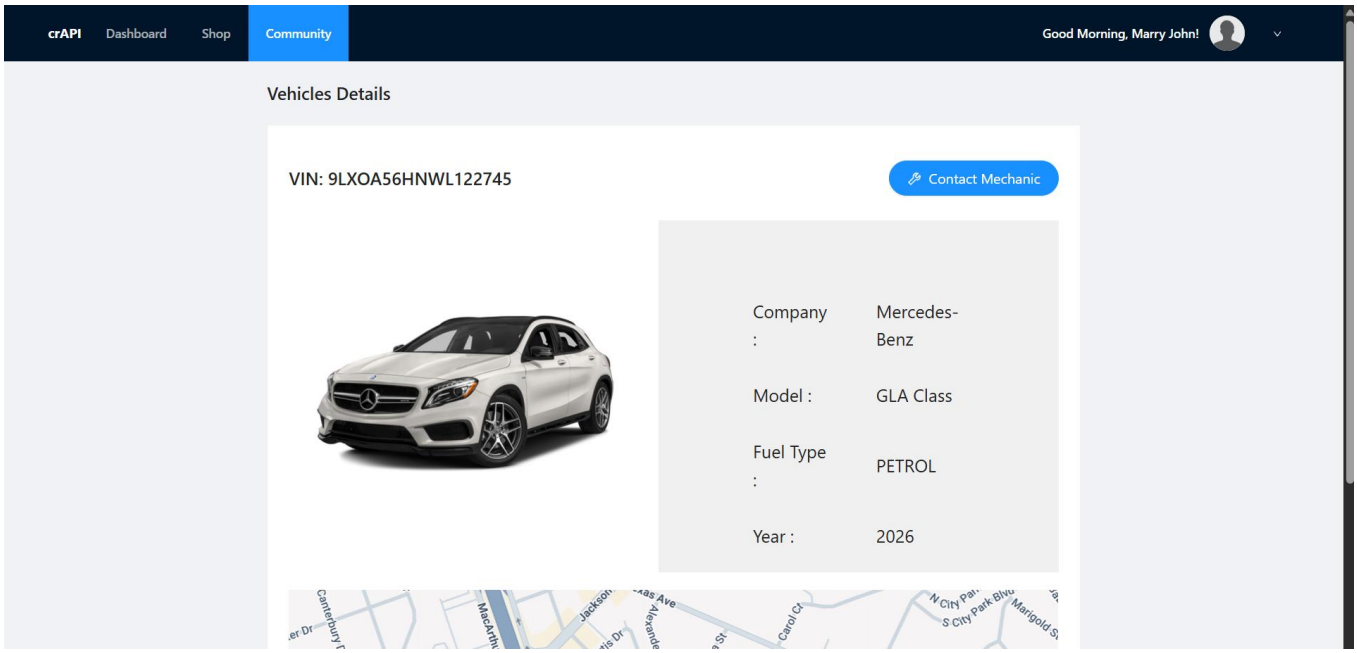

Figure 3.1 — Community tab posts as seen in the browser (filtered view)

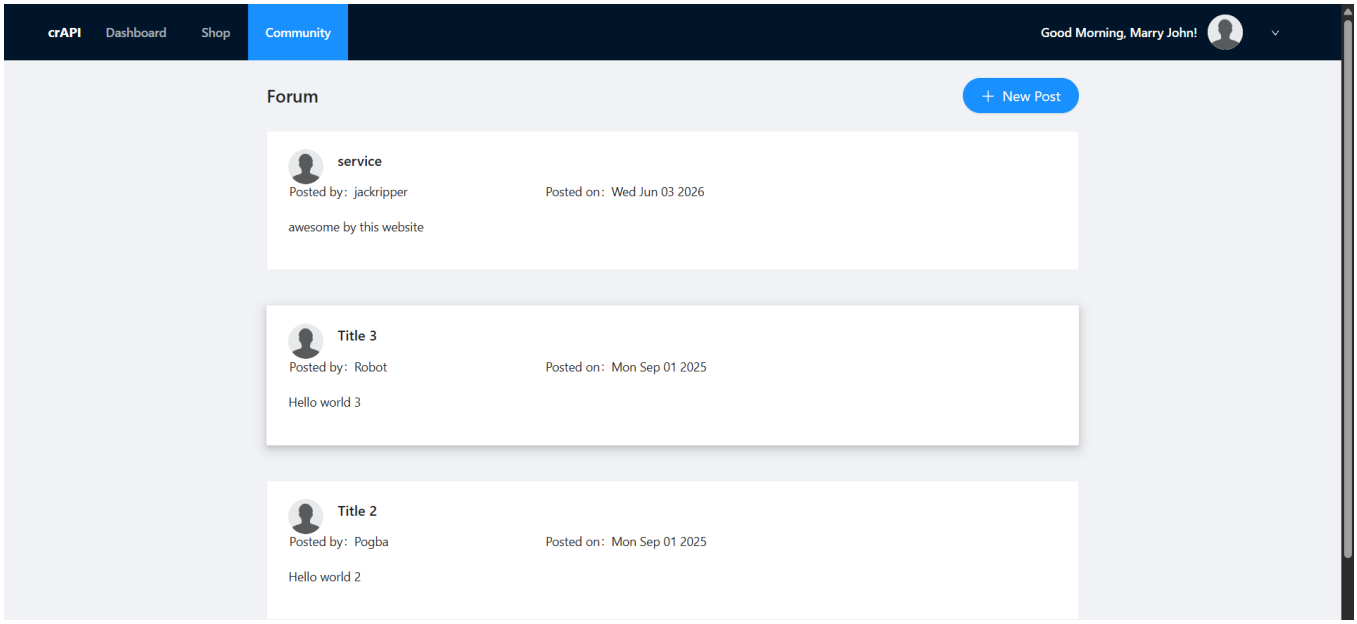

Figure 3.2 — Community feed request

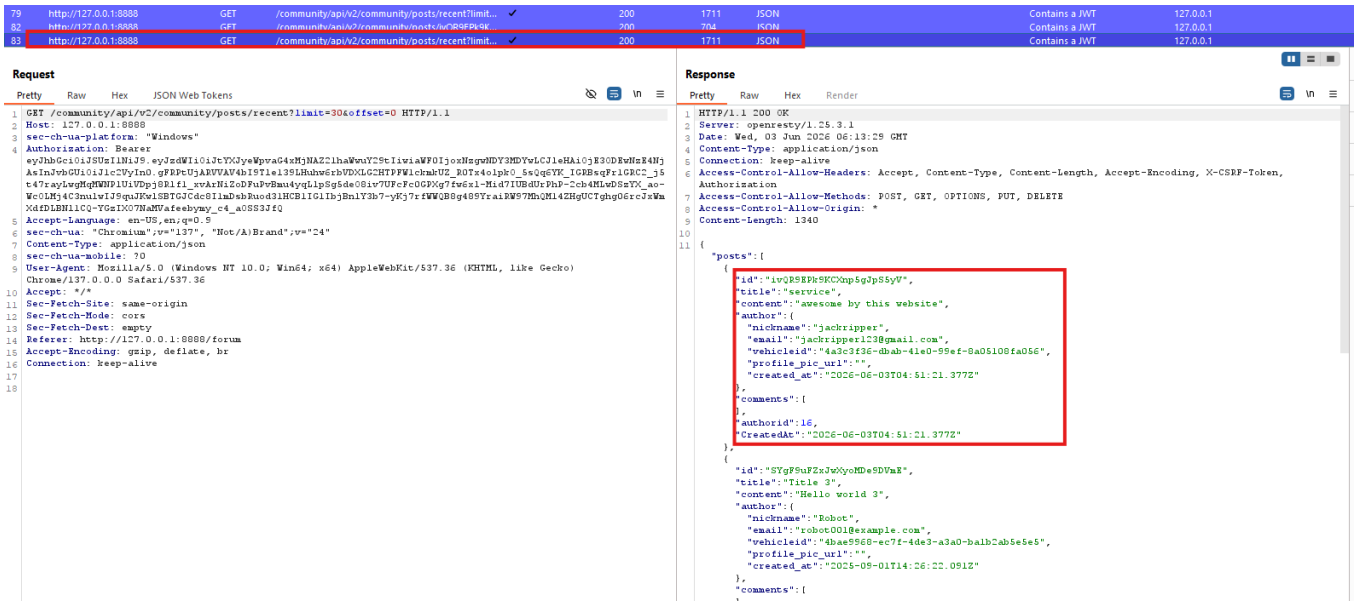

Figure 3.3 — Community feed request captured in Burp Suite & Raw API response exposing internal user IDs, emails, and vehicle IDs (Excessive Data Exposure confirmed)

Confirmed Vulnerability

The unfiltered API response exposed excessive user attributes including internal user IDs, email addresses, and vehicle IDs for all community users. The browser JavaScript layer was suppressing this data from view, but it was fully present in the raw API response — a textbook's Excessive Data Exposure vulnerability.

Remediation

- Implement **field-level authorization checks** to ensure users can only view and modify properties they are explicitly permitted to access.
- Use **allowlists (whitelists)** to define which object properties can be read or updated, rather than accepting all client-supplied fields.
- Prevent **Excessive Data Exposure** by returning only the data required for the intended functionality and filtering sensitive fields from API responses.
- Mitigate **Mass Assignment** vulnerabilities by explicitly mapping and validating user input instead of automatically binding request parameters to backend objects.
- Conduct regular **security testing and code reviews** to identify unauthorized property access, excessive data exposure, and improper field update mechanisms.

API4

API4:2023

Unrestricted Resource Consumption

Overview

Unrestricted Resource Consumption is a vulnerability that occurs when an API does not enforce adequate limits on the resources a client can consume. Resources such as CPU, memory, storage, network bandwidth, database connections, and third-party service quotas can be exhausted when users are allowed to make excessive requests or perform resource-intensive operations without restrictions. This weakness often results from the absence of rate limiting, request throttling, payload size restrictions, pagination controls, or execution time limits. As a result, attackers can abuse API functionality by repeatedly sending requests, uploading large files, or triggering expensive backend processes.

The impact of this vulnerability can be significant, ranging from degraded application performance to complete Denial of Service (DoS) conditions that prevent legitimate users from accessing the service. In cloud-based environments, excessive resource consumption may also lead to substantial financial costs due to automatic scaling of infrastructure and increased usage of paid services. Successful exploitation can disrupt business operations, reduce system availability, and create unnecessary operational expenses. Therefore, APIs should implement appropriate resource controls and monitoring mechanisms to ensure fair usage and protect system stability.

Exploitation Steps

- Log in and navigate to the Contact Mechanic tab. Submit a service request and capture the request in Burp Suite.
- Inspect the captured request. Identify the repeat_request boolean and num_responses integer parameters in the request body.
- Forward the request to Repeater. Set repeat_request to true and change the repeat count to 10000.
- Send the modified request and observe the response.

Evidence

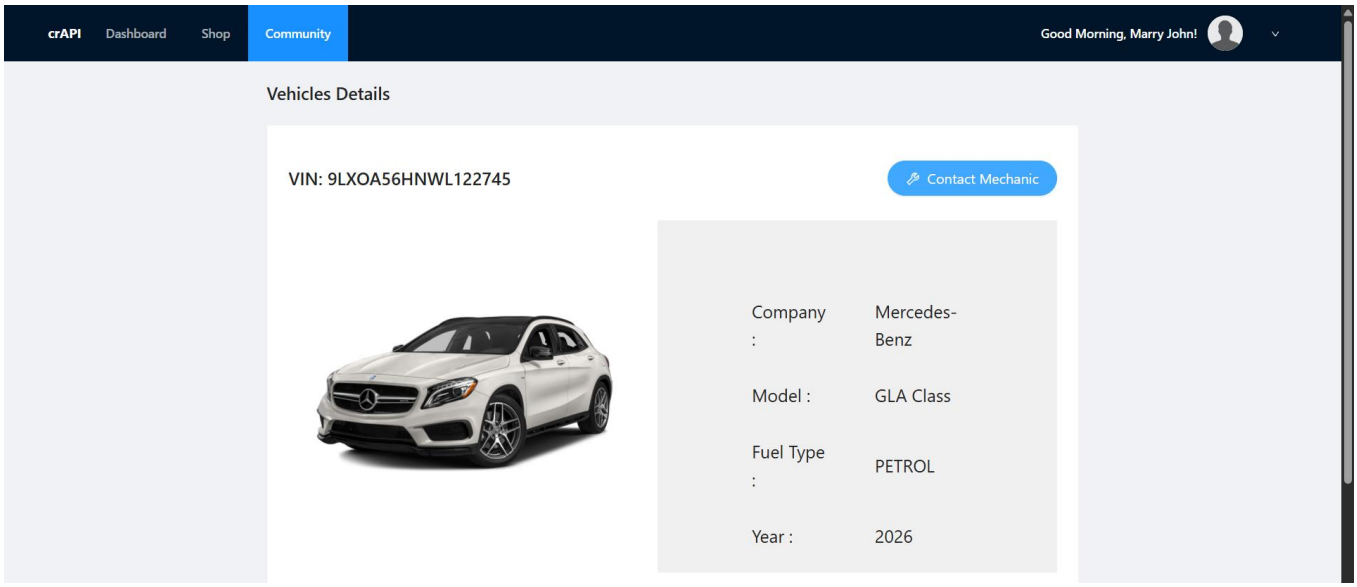

Figure 4.1 — Contact Mechanic tab in the crAPI application

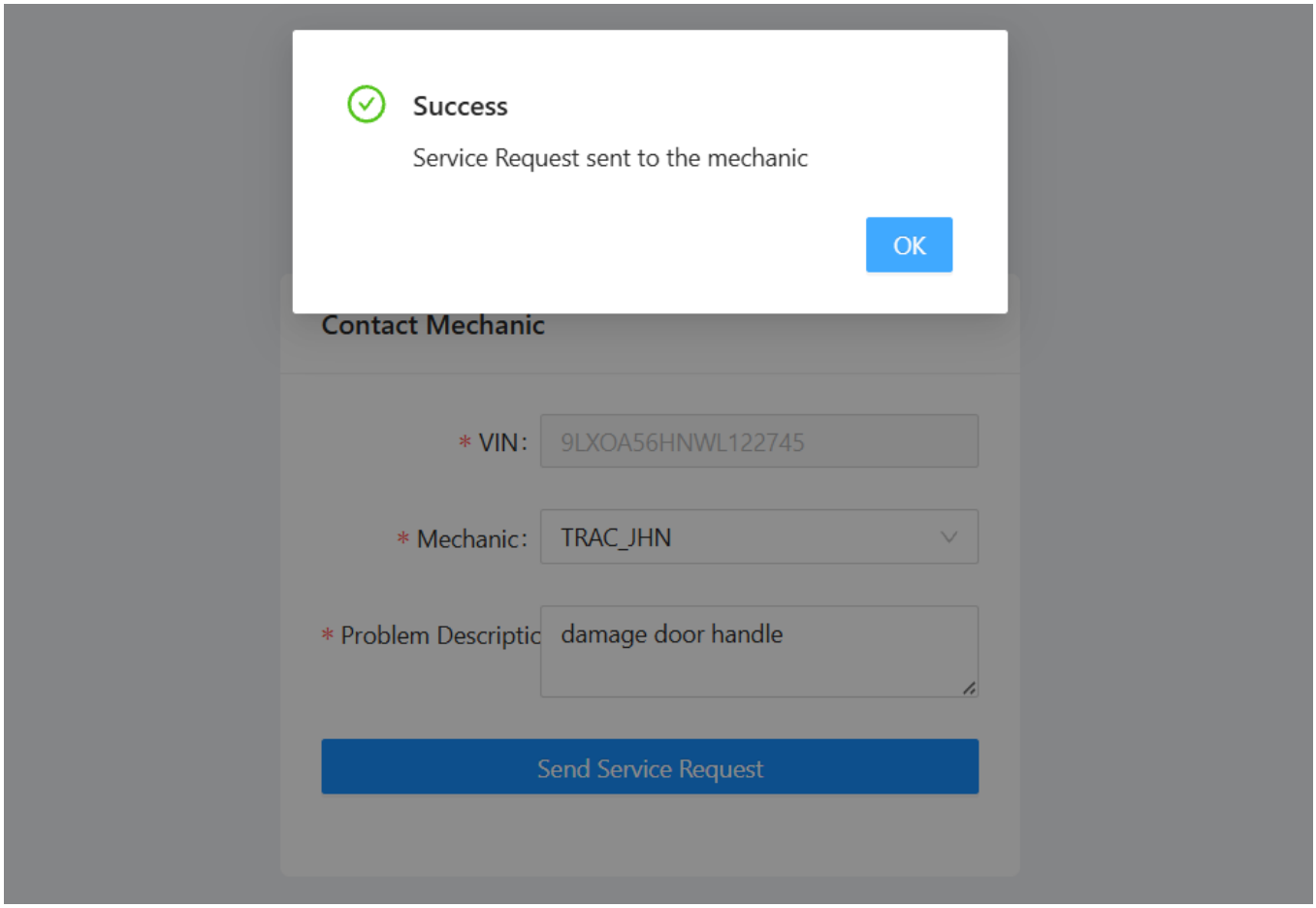

Figure 4.2 — Service request form filled and submitted

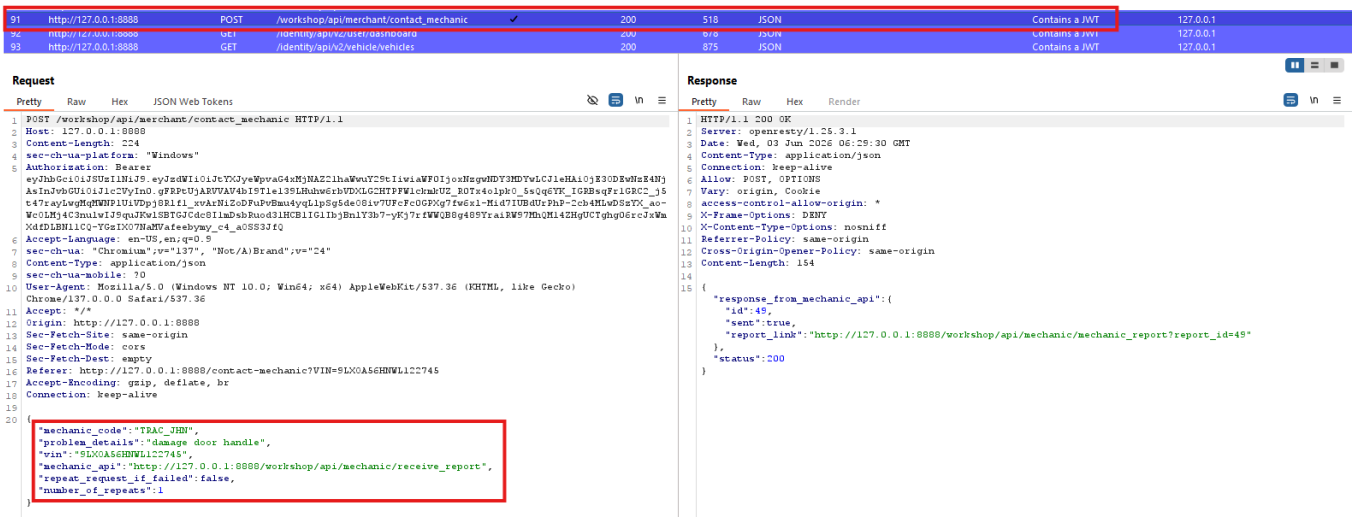

Figure 4.3 — Captured request and change the parameters value of "repeat_request_if_failed: true" and "number_of_repeats:10000"

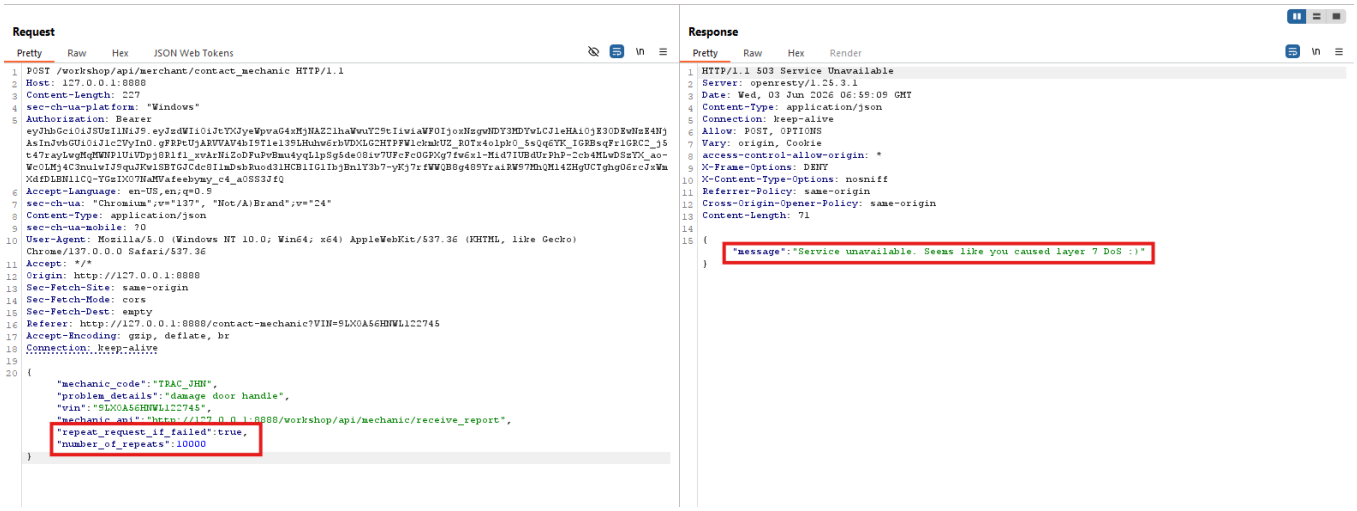

Figure 4.4 — HTTP 503 'Service Unavailable' response confirms Layer 7 DoS attack successful

Confirmed Vulnerability

The server responded with HTTP 503 and the message: "Service unavailable. Seems like you caused layer 7 DoS." This confirms that the API endpoint lacks input validation on resource-consuming parameters, enabling a denial-of-service attack with a single modified request.

Remediation

- Implement **rate limiting and request throttling** to restrict the number of requests a client can make within a specified time.
- Enforce **resource usage limits** such as maximum payload sizes, file upload restrictions, query complexity limits, and execution timeouts.

- Use **pagination and result-size restrictions** to prevent clients from requesting excessive amounts of data in a single API call.
- Monitor and log resource consumption to detect abnormal usage patterns and respond to potential abuse or denial of service attempts.
- Configure quotas and automated safeguards to control the consumption of cloud resources, storage, bandwidth, and third-party service integrations.

API5

API5:2023

Broken Function Level Authorization

Overview

Broken Function Level Authorization (BFLA) is a vulnerability that occurs when an API fails to properly enforce authorization checks on specific functions or actions, allowing users to perform operations beyond their intended privileges. While the application may correctly authenticate users and restrict access to certain interface elements, it relies on the client-side application or user interface to hide privileged functionality instead of enforcing access control on the server. As a result, unauthorized users can directly access sensitive API endpoints by modifying requests or interacting with endpoints that are not exposed through the user interface.

This vulnerability commonly arises when administrative or privileged functions, such as user management, account deletion, role modification, configuration changes, or access to administrative panels, are not protected by adequate server-side authorization mechanisms. Attackers can identify these endpoints through API documentation, traffic analysis, endpoint enumeration, or by inspecting client-side code. Once discovered, they can send crafted API requests directly to the vulnerable endpoints and execute actions reserved for administrators or higher-privileged users. Successful exploitation can lead to privilege escalation, unauthorized modification of critical system settings, data manipulation, account takeover, and complete compromise of application functionality. Because the vulnerability affects access to business functions rather than individual resources, it can have severe consequences for the security and integrity of the entire application.

Exploitation Steps

- Log in and navigate to the profile section. Upload a video file.
- Capture the video rename request in Burp Suite and forward it to Repeater.
- Change the HTTP method from PUT to OPTIONS to enumerate permitted methods on the endpoint.
- The response reveals: GET, HEAD, DELETE, POST, OPTIONS.
- Modify the request to use the DELETE method and observe the “This is an admin function” message.
- Change the URL path segment from user to admin and resubmit the DELETE request.

Evidence

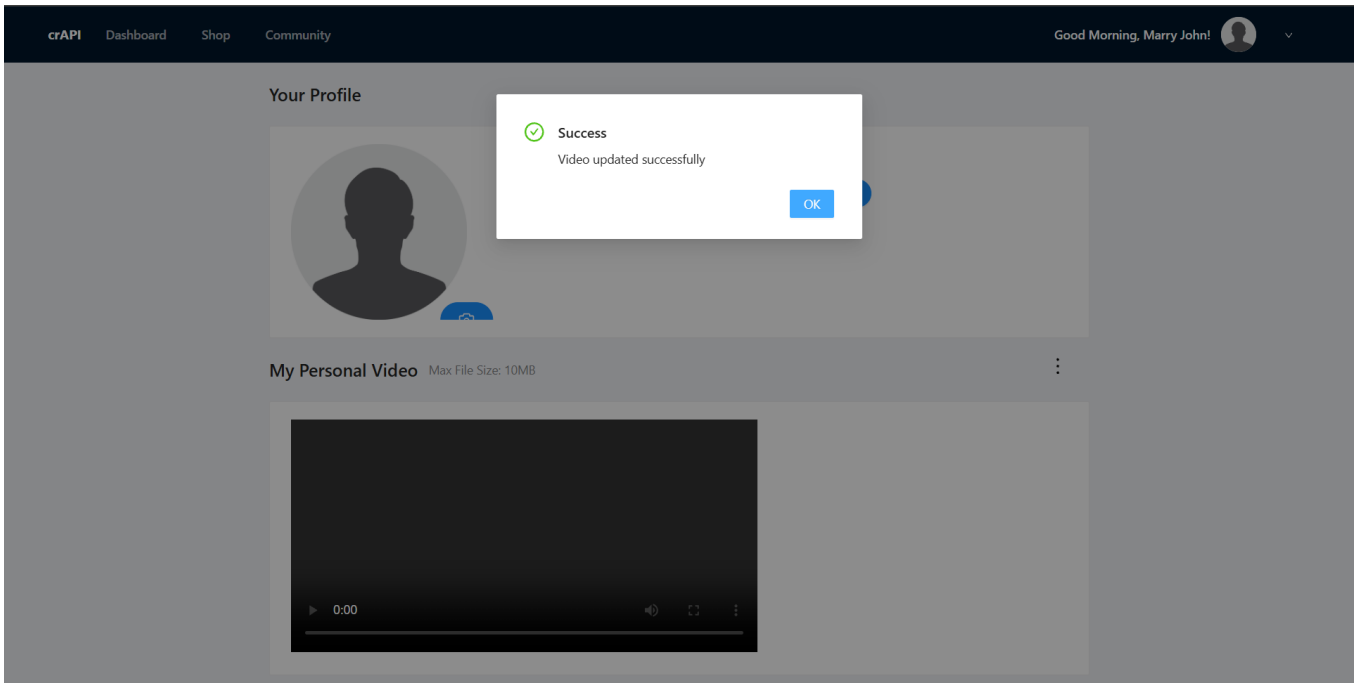

Figure 5.1 — Profile section with video upload option, video uploaded

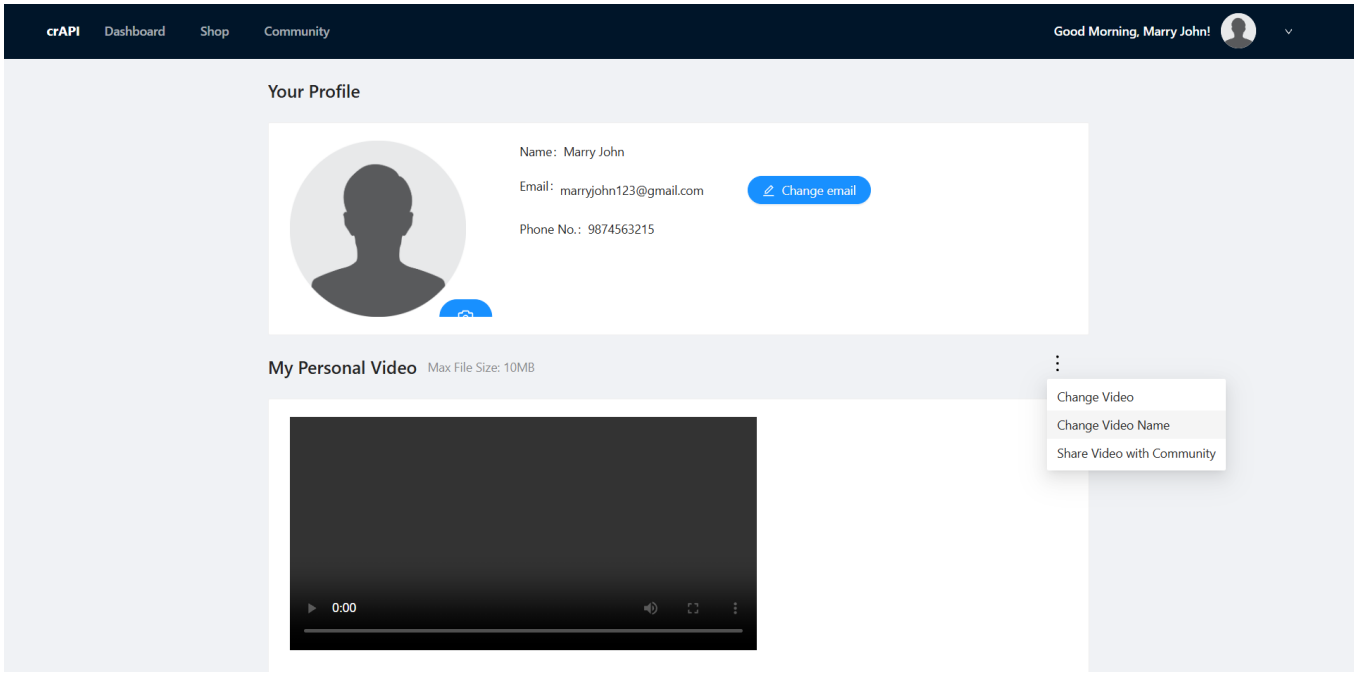

Figure 5.2 — Rename the video from the profile

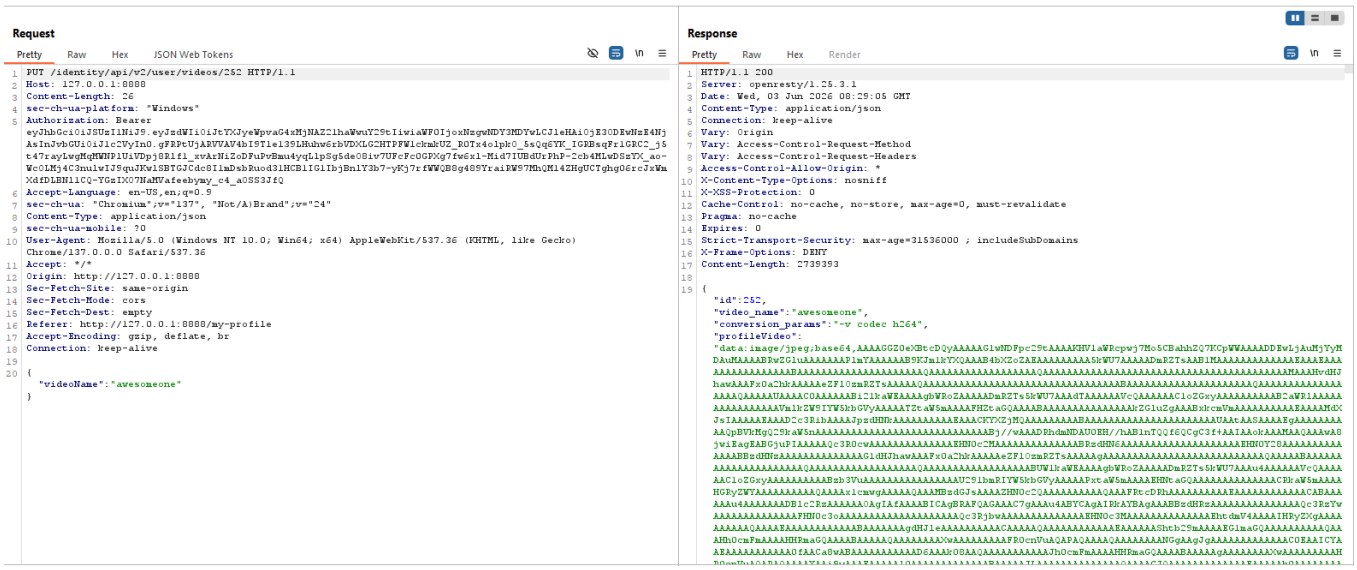

Figure 5.3 — Request captured in Burp Suite

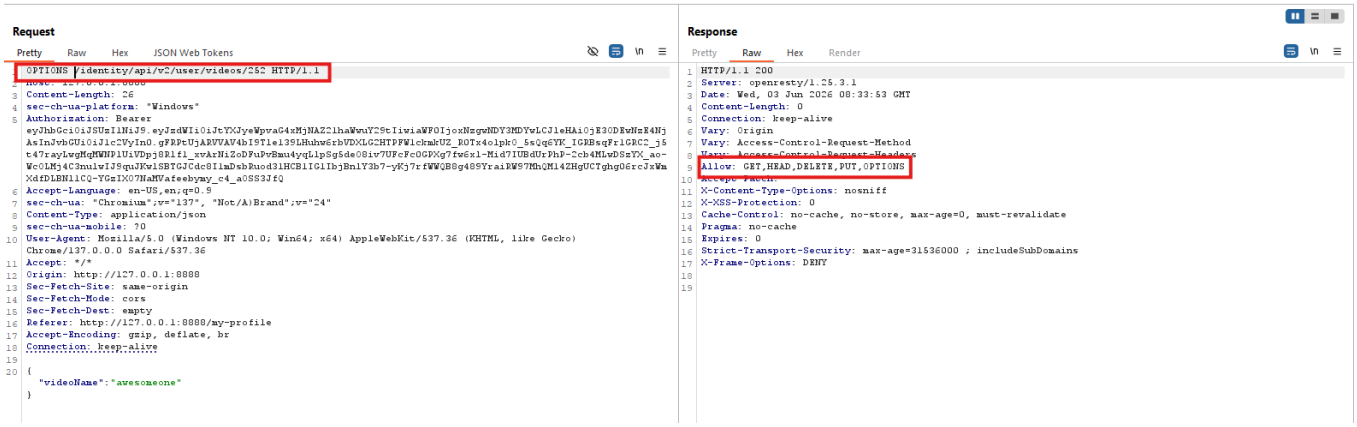

Figure 5.4 — OPTIONS method response revealing DELETE as an available operation

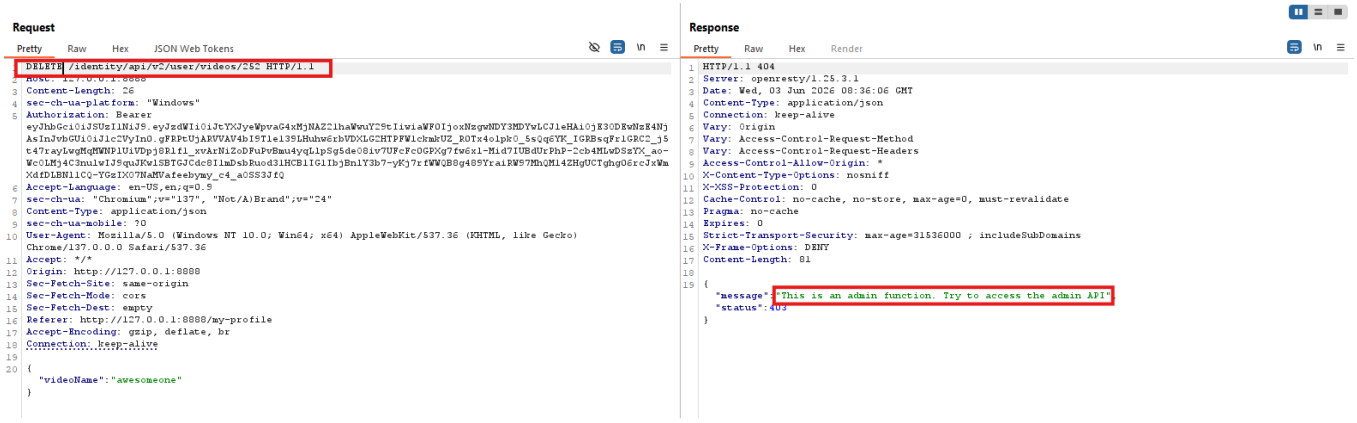

Figure 5.5 — DELETE method returns “This is an admin function” message

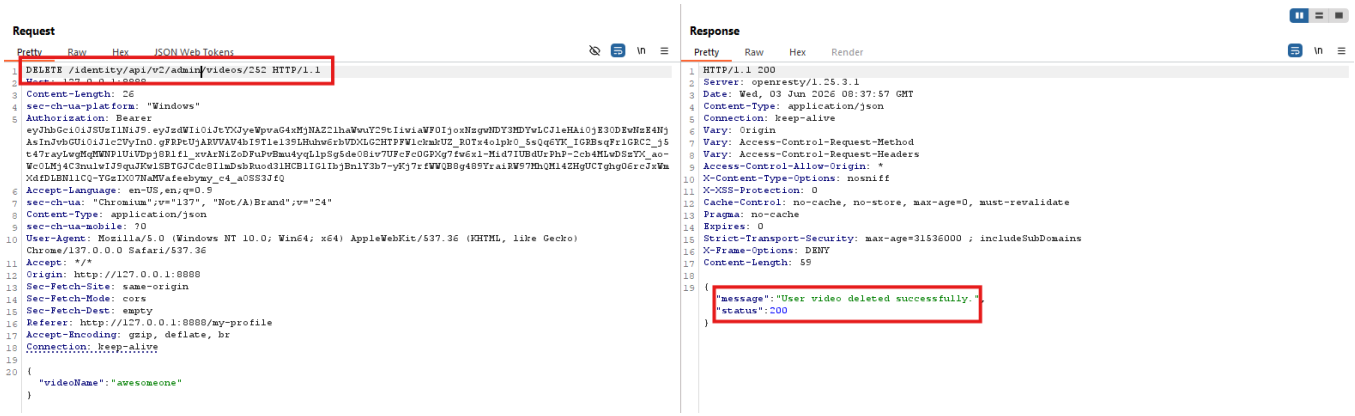

Figure 5.6 — Changing path from /user/ to /admin/ & Video successfully deleted using standard user account (BFLA confirmed)

Confirmed Vulnerability

The video was successfully deleted using a user-level account by modifying the API path to reference the admin endpoint. The server performed no role validation on the request, confirming a Broken Function Level Authorization vulnerability.

Remediation

- Enforce **server-side authorization checks** for every API function and endpoint, regardless of whether it is visible in the user interface.
- Implement **role-based access control (RBAC)** or attribute-based access control (ABAC) to ensure users can only perform actions permitted for their role.
- Deny access by default and explicitly grant permissions only to authorized users for sensitive or administrative functions.
- Regularly review and test API endpoints to identify unauthorized access to privileged functions through penetration testing and security assessments.
- Maintain a centralized authorization mechanism to ensure consistent access control enforcement across all API endpoints and business functions.

API6

API6:2023

Unrestricted Access to Sensitive Business Flows

Overview

This vulnerability occurs when an API correctly authenticates users and enforces authorization controls but fails to limit how often or at what scale a legitimate business function can be performed. Unlike traditional security flaws that result from coding errors or misconfigurations, this vulnerability stems from weaknesses in business logic design. The API allows users to repeatedly execute valid actions without sufficient safeguards, enabling attackers to abuse intended functionality for malicious purposes. Because the requests themselves are legitimate and follow normal application workflows, detecting and preventing such abuse can be particularly challenging.

Attackers typically exploit this weakness by using automation tools, scripts, or bots to perform business operations at a speed and volume far beyond what a normal user could achieve. Examples include repeatedly redeeming promotional offers, rapidly purchasing limited-availability products, generating excessive transactions, abusing reward or loyalty programs, or submitting large numbers of requests that manipulate business processes for financial gain. Successful exploitation can result in revenue loss, resource depletion, unfair advantages for attackers, disruption of business operations, and damage to customer trust. To prevent this type of abuse, organizations must implement business-specific controls that limit the frequency, volume, and impact of sensitive operations while still allowing legitimate users to access the intended functionality.

Exploitation Steps

- Purchase an item (e.g., a wheel for $10) via the shop and capture the order request in Burp Suite.
- Note that the account balance decreases from $100 to $90 after a legitimate purchase.
- Forward the order request to Repeater. Modify the quantity parameter from 1 to −1000.
- Send the modified request and observe the response.

Evidence

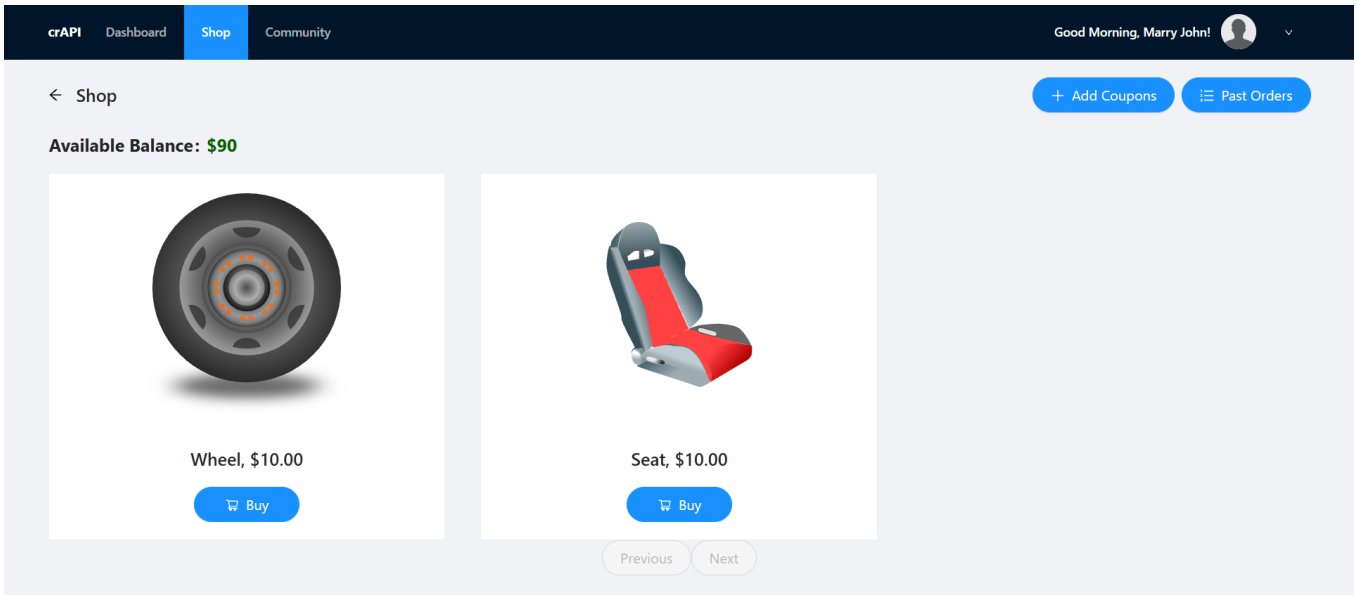

Figure 6.1 — Shop page showing available items

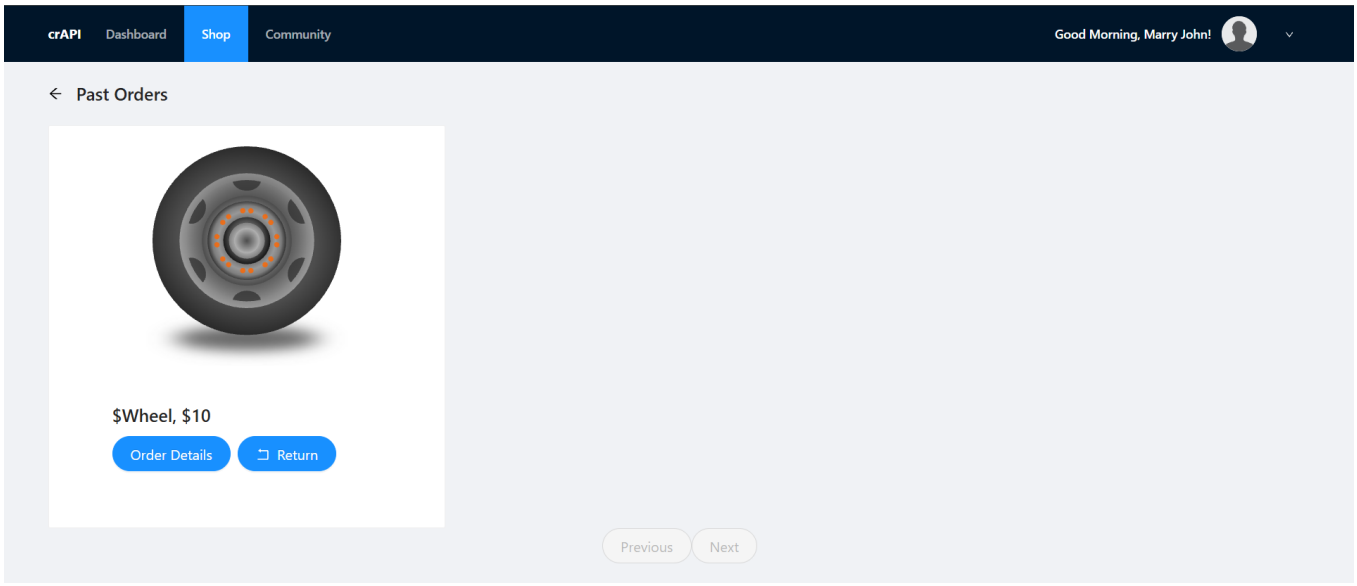

Figure 6.2 — Past orders page; balance reduced to $90 after $10 purchase

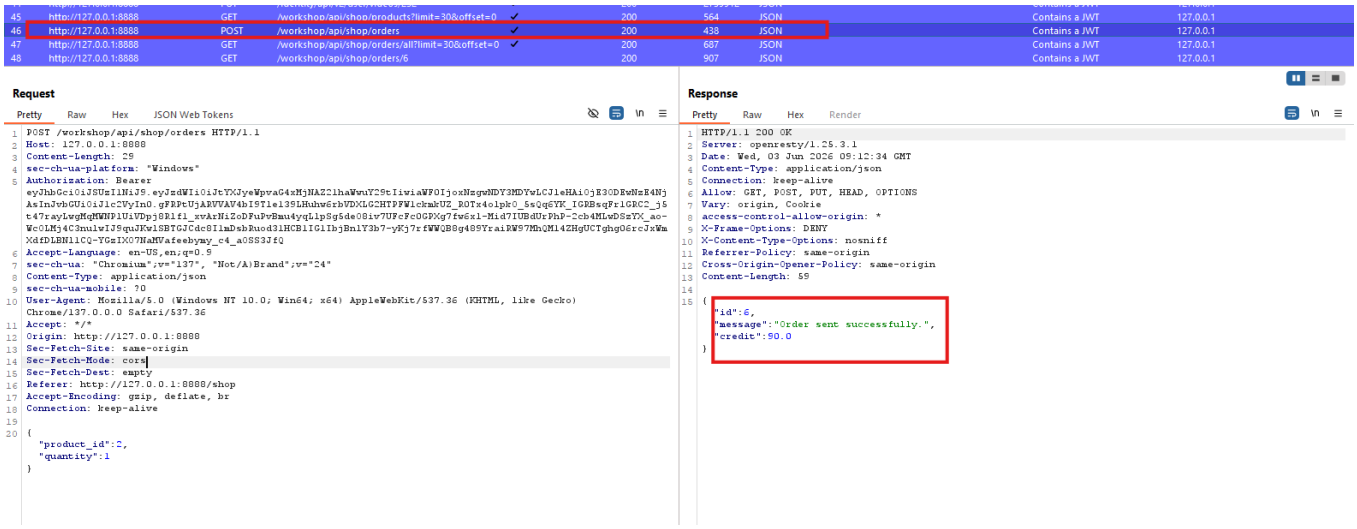

Figure 6.3 — Purchase request captured in Burp Suite

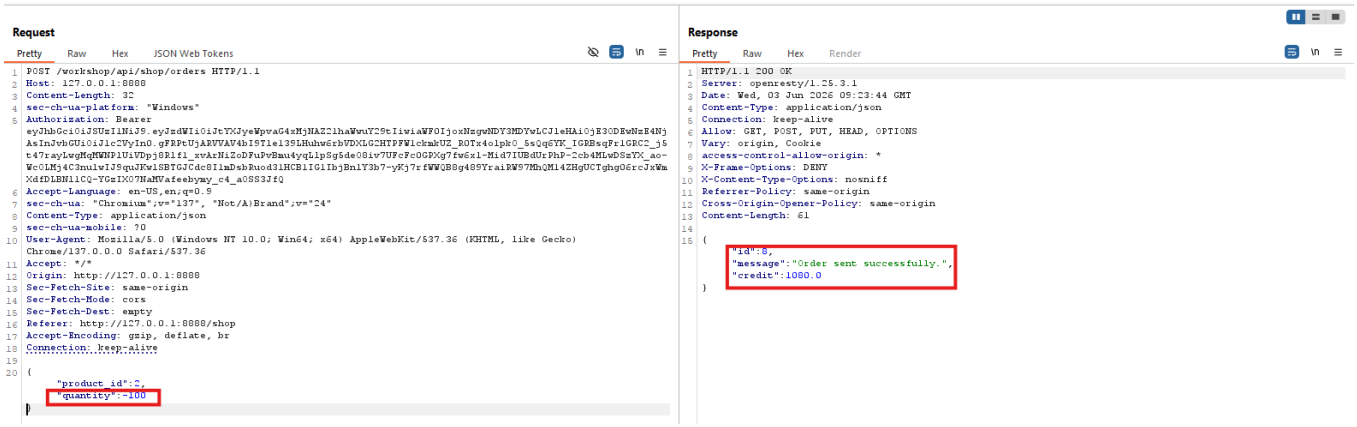

Figure 6.4 — Quantity modified to −100 in Repeater

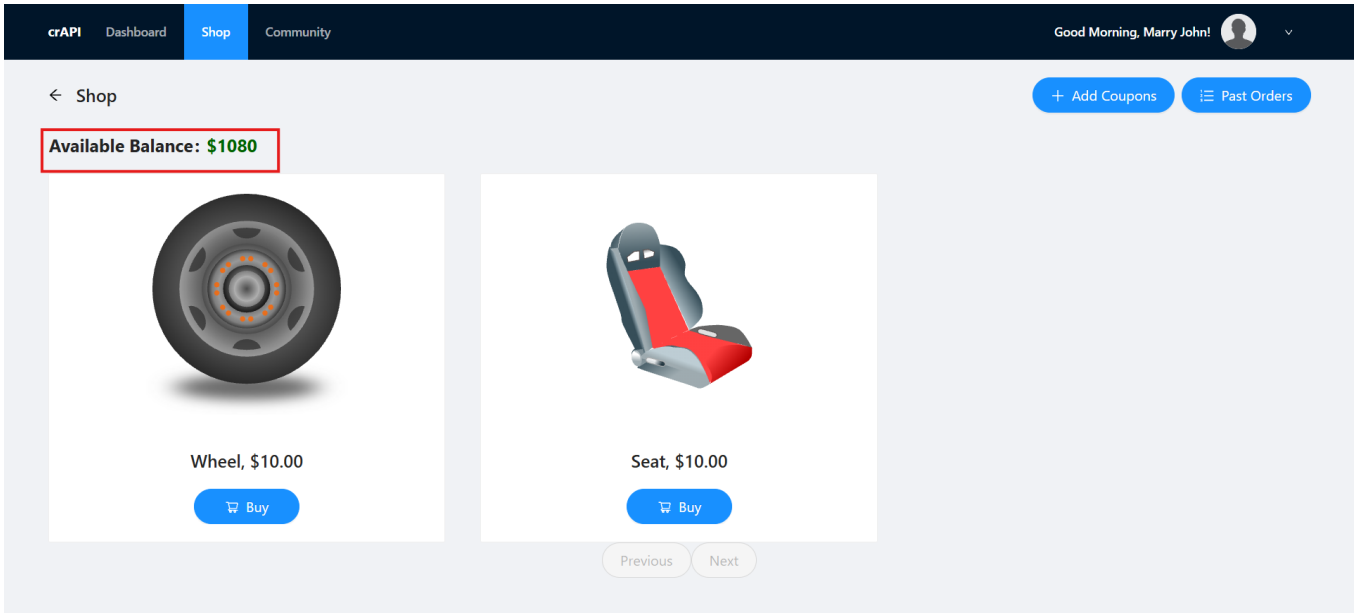

Figure 6.5 — Account balance increases after negative quantity — purchase endpoint abused as refund endpoint (confirmed)

Confirmed Vulnerability

The server accepted a negative quantity value and credited the account balance instead of debiting it, effectively converting a purchase endpoint into an unlimited refund endpoint. This confirms that the API performs no business logic validation on purchase quantities.

Remediation

- Implement **rate limiting and transaction throttling** on sensitive business operations to prevent automated abuse and excessive execution.
- Enforce **business-specific limits**, such as maximum purchases, redemptions, transfers, or requests per user within a defined time period.
- Use **CAPTCHA, bot detection, or human verification mechanisms** to reduce the risk of automated attacks against critical workflows.
- Monitor and analyze business transactions for unusual patterns, high-frequency activity, or other indicators of abuse.
- Conduct regular **business logic security reviews and penetration testing** to identify and address weaknesses that could be exploited through automation.

API7 API7:2023

Server-Side Request Forgery (SSRF)

Overview

Server-Side Request Forgery (SSRF) is a vulnerability that occurs when a web application or API retrieves remote resources based on user-supplied input, such as a URL, without properly validating, sanitizing, or restricting the destination of the request. In these situations, the server acts as a proxy and sends requests on behalf of the user. If adequate controls are not in place, an attacker can manipulate the request target and force the server to communicate with unintended systems, including internal services that are not directly accessible from the internet.

SSRF is particularly dangerous because it allows attackers to leverage the trust and network access privileges of the vulnerable server. By crafting malicious requests, attackers can interact with internal infrastructure, scan private network ranges, access internal APIs, retrieve sensitive information from cloud metadata services, or communicate with external systems under their control. In cloud environments, SSRF vulnerabilities are often exploited to access metadata endpoints that may expose credentials, access tokens, or configuration details. Successful exploitation can lead to information disclosure, internal network reconnaissance, privilege escalation, unauthorized access to backend services, and, in some cases, complete compromise of the affected environment. Because the requests originate from the trusted server rather than the attacker, traditional network security controls may not detect or prevent such activity, making SSRF a significant risk in modern web applications and APIs.

Exploitation Steps

- Navigate to the Contact Mechanic tab and submit a service request.
- Capture the request in Burp Suite and inspect it. Observe that the request is forwarded internally to a workshop service endpoint.
- Forward the request to Repeater. Replace the internal service URL with an external URL (e.g., http://google.com/).
- Send the modified request and observe the response.

Evidence

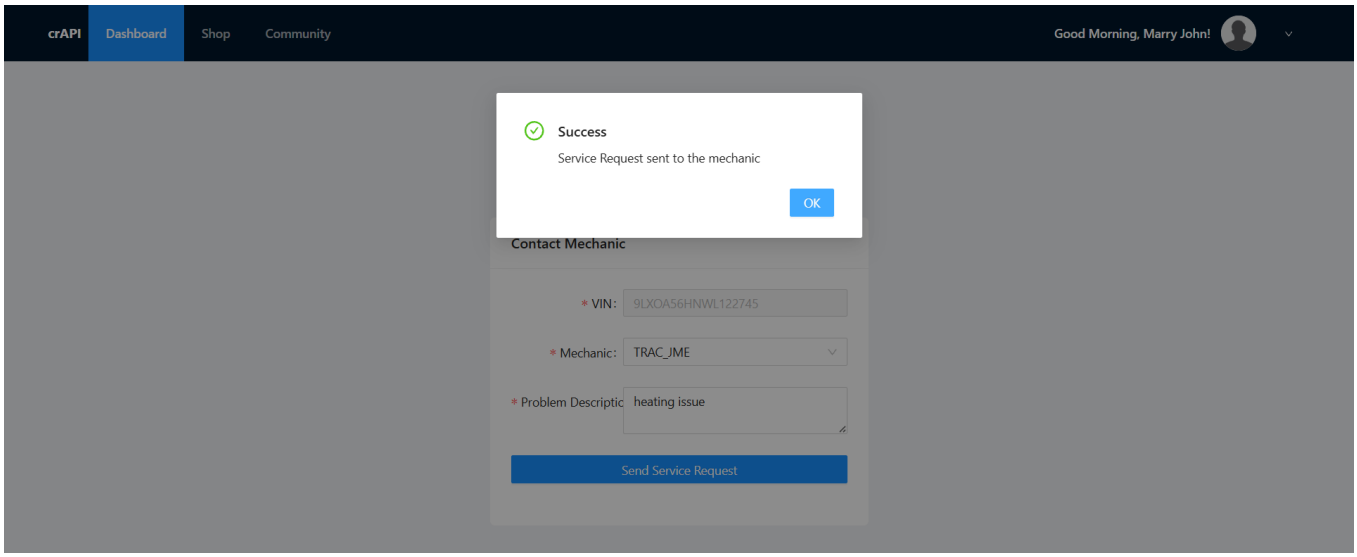

Figure 7.1 — Contact Mechanic form with service request

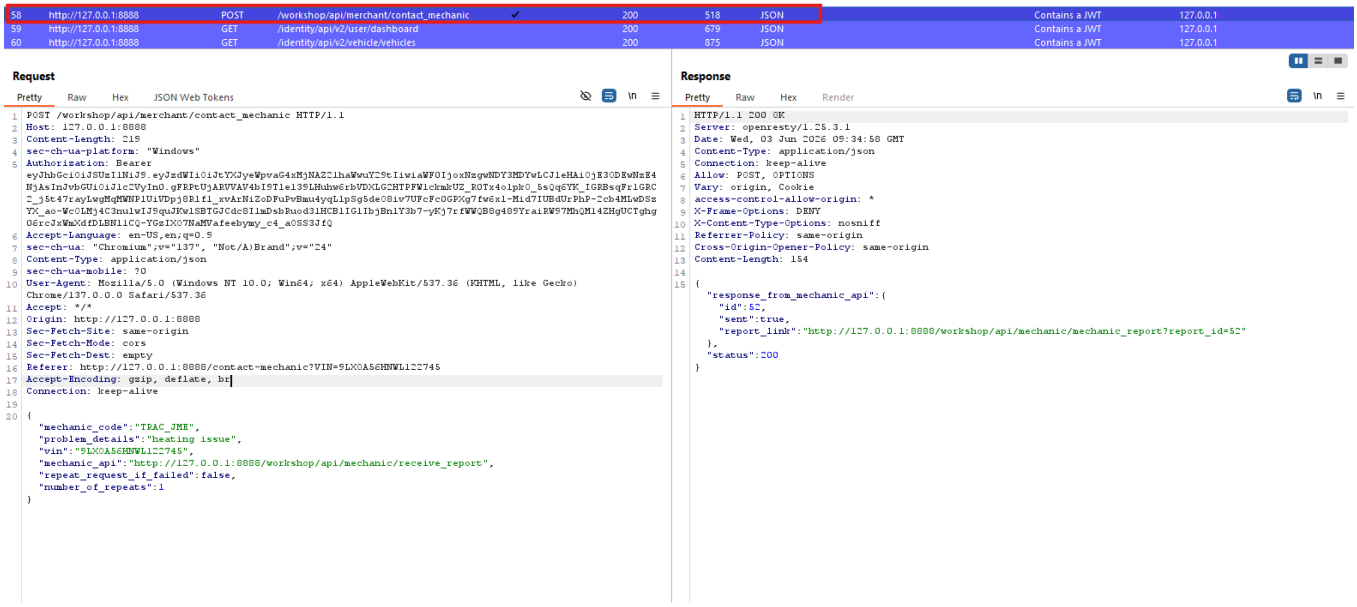

Figure 7.2 — Request captured showing internal workshop service URL

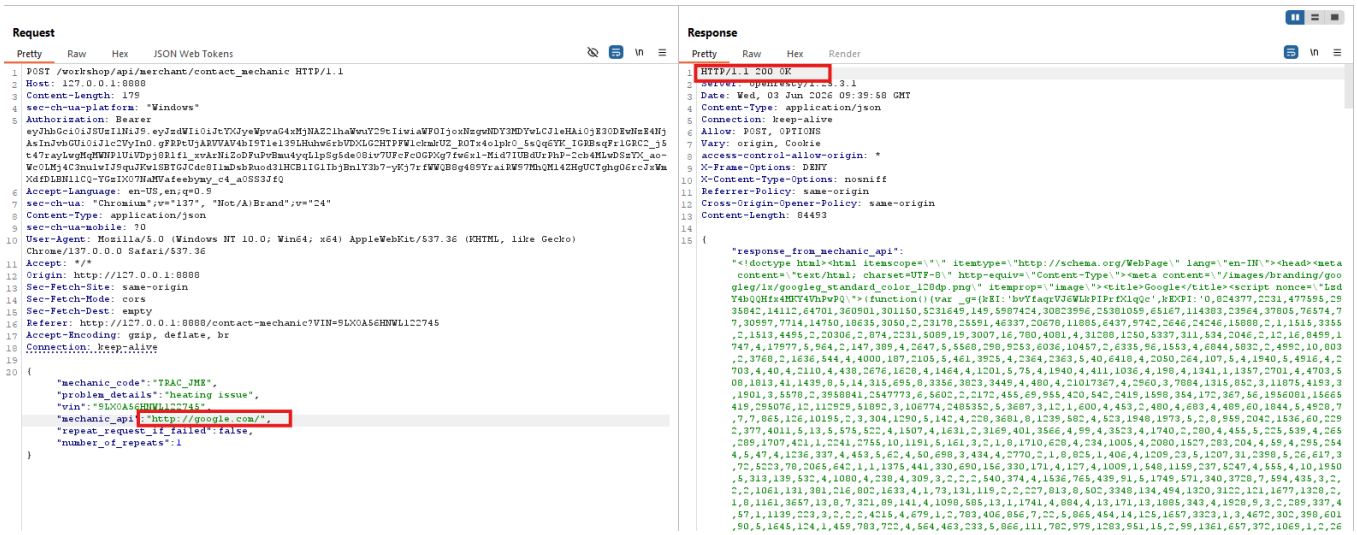

Figure 7.3 — HTTP 200 response after substituting external URL confirms SSRF vulnerability

Confirmed Vulnerability

The server returned an HTTP 200 response after fetching the externally supplied URL, confirming that the application performs no URL allowlist validation. This SSRF vulnerability could be leveraged to probe internal services, bypass firewalls, or access cloud metadata endpoints.

Remediation

- Validate and sanitize all user-supplied URLs, allowing requests only to trusted and explicitly approved destinations through an **allowlist** approach.
- Block access to internal IP ranges, localhost addresses, cloud metadata services, and other sensitive network resources that should not be reachable from user-controlled requests.
- Implement strict network segmentation and firewall rules to restrict the server's ability to communicate with unnecessary internal or external systems.
- Disable automatic URL redirects and validate the final destination of requests to prevent bypassing security controls.
- Monitor and log outbound server requests to detect suspicious activity, unauthorized destinations, and potential SSRF exploitation attempts.

API8

API8:2023

Security Misconfiguration

Overview

Security Misconfiguration is a vulnerability that arises when applications, APIs, servers, databases, cloud services, or supporting infrastructure are deployed with insecure settings or are not properly hardened before being placed into production. It is one of the most common security weaknesses because it can occur at any layer of the technology stack and often results from using default configurations, incomplete setup procedures, unnecessary enabled features, or a lack of consistent security management practices. Even when application code is secure, misconfigurations can create opportunities for attackers to gain unauthorized access or obtain sensitive information.

Examples of security misconfiguration include default usernames and passwords, improperly configured access controls, missing or insecure HTTP security headers, exposed administrative interfaces, open cloud storage buckets, verbose error messages that reveal internal system details, outdated software components, unnecessary services running in production environments, and excessive permissions assigned to users or applications. Attackers can exploit these weaknesses to gather information about the system, bypass security controls, access sensitive data, or compromise critical infrastructure. The impact may include information disclosure, unauthorized access, service disruption, and increased exposure to other attacks. Because configurations can change over time, organizations should implement secure configuration standards, perform regular security reviews, and continuously monitor systems to ensure that secure settings are maintained throughout the application lifecycle.

Common Examples

Default credentials left unchanged • Overly permissive CORS policies • Stack traces and debug information exposed in error responses • Unnecessary HTTP methods enabled on endpoints • Missing security headers (e.g., Content-Security-Policy, X-Frame-Options)

Remediation

- Establish and maintain **secure configuration baselines** for applications, servers, databases, cloud services, and network devices before deployment.
- Remove or disable **default accounts, unnecessary services, unused features, and sample applications** that are not required in production environments.
- Configure appropriate **security headers, access controls, and permissions** to protect sensitive resources and reduce the attack surface.
- Regularly **patch and update** operating systems, applications, frameworks, and third-party components to address known vulnerabilities.

API9

API9:2023

Improper Inventory Management

Overview

Improper Inventory Management is a vulnerability that occurs when an organization fails to maintain a complete and accurate inventory of its APIs, including their versions, environments, endpoints, and associated documentation. As APIs evolve over time, new versions are introduced, older versions are deprecated, and development, testing, or staging environments may remain accessible. Without proper visibility and management, organizations can lose track of these assets, creating security gaps that attackers can exploit.

This vulnerability commonly arises when outdated API versions remain active, undocumented endpoints are exposed, or non-production environments are left accessible from the internet. Forgotten staging servers, legacy APIs, and shadow APIs often do not receive the same security updates, monitoring, authentication controls, or access restrictions as actively maintained production systems. Attackers can discover these overlooked assets through reconnaissance, endpoint enumeration, or public documentation and use them as alternative entry points into the environment. Successful exploitation can lead to unauthorized access, information disclosure, bypass of security controls, and compromise of sensitive systems. Effective API inventory management is therefore essential to ensure that all API assets are properly secured, monitored, and maintained throughout their lifecycle.

Exploitation Steps

- Using the email harvested earlier, initiate a password reset and capture the OTP verification request in Burp Suite.
- Observe the target endpoint: POST /identity/api/auth/v3/check-otp. The v3 version enforces an OTP attempt limit, returning HTTP 500 after excessive failures.
- Modify the API version in the URL path from v3 to v2 to query the older, unpatched endpoint.
- Forward the modified request to Intruder and configure a Sniper attack with a numeric payload (0000–9999).

Evidence

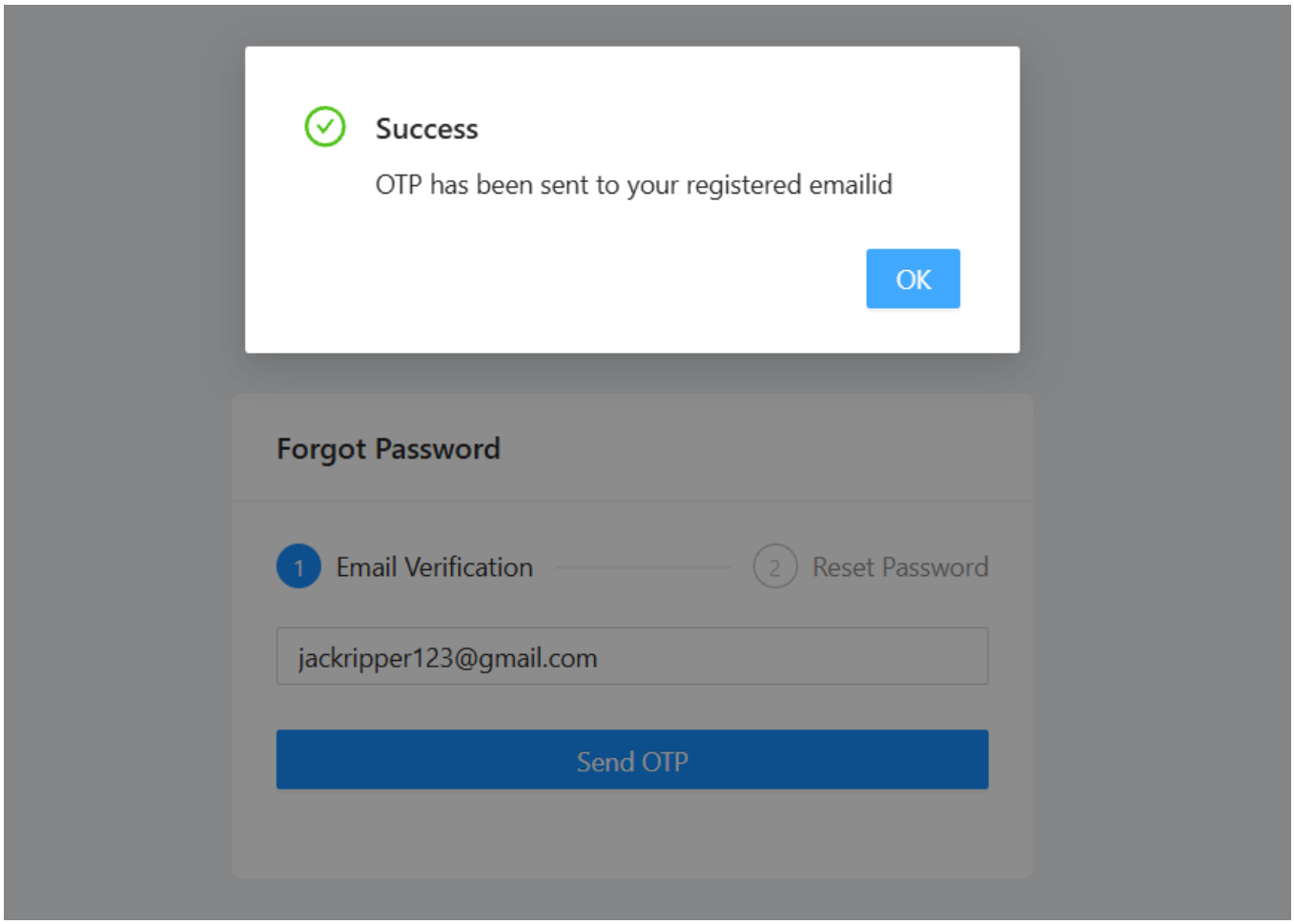

Figure 9.1 — Password reset request for target account

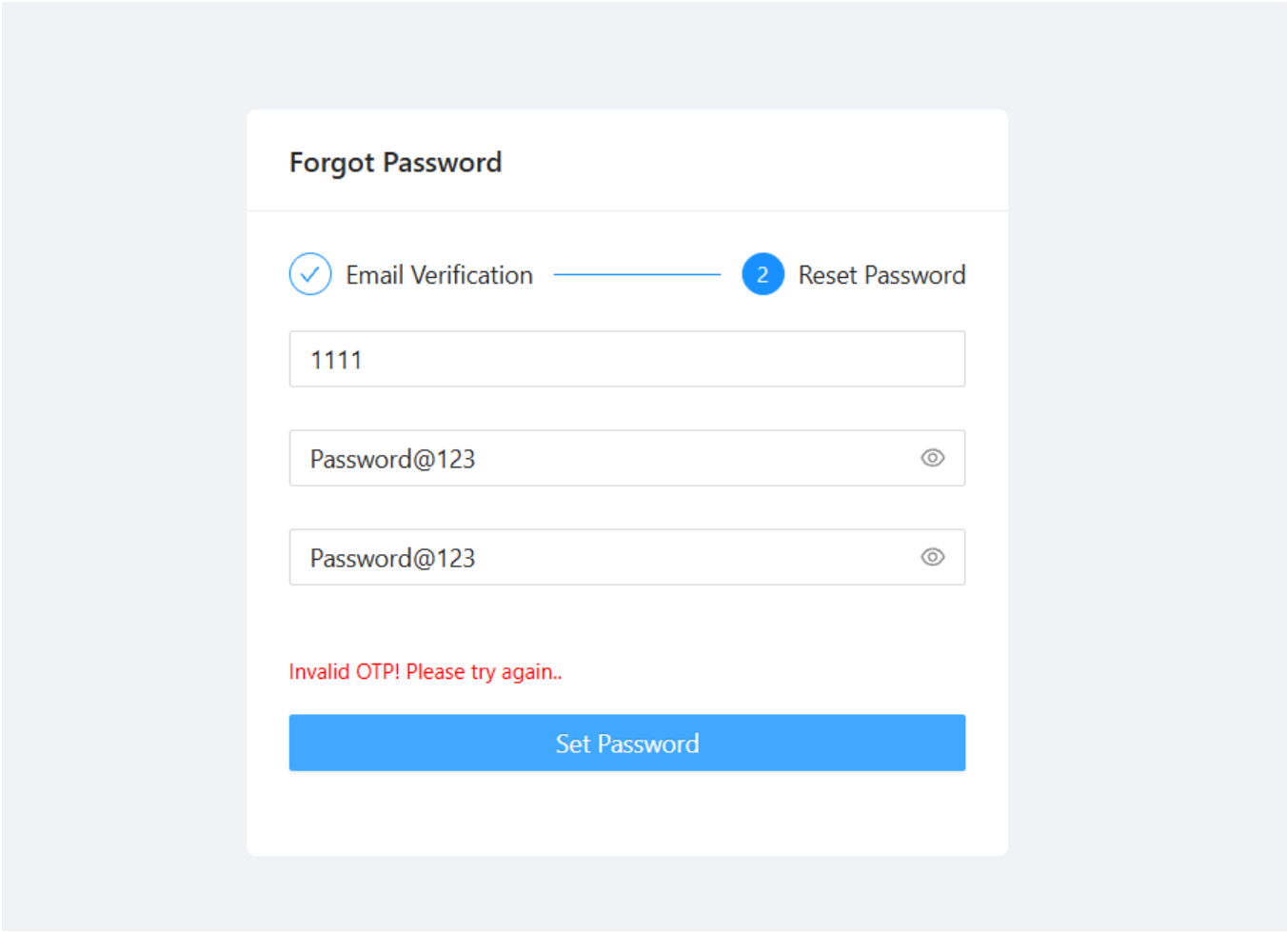

Figure 9.2 — OTP check request to v3 endpoint (rate-limited)

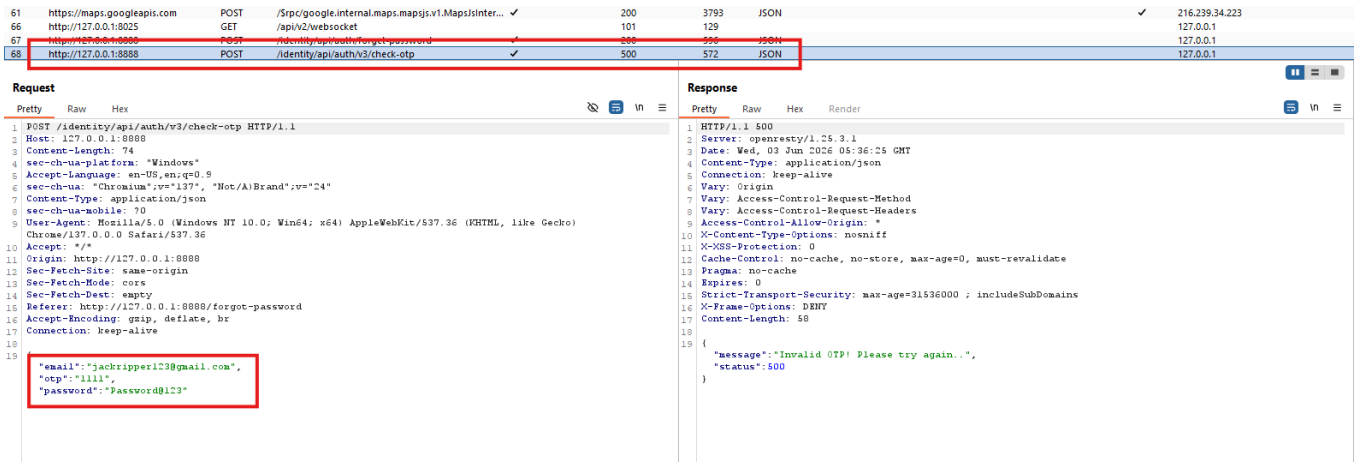

Figure 9.3 — API version changed from v3 to v2 in request path

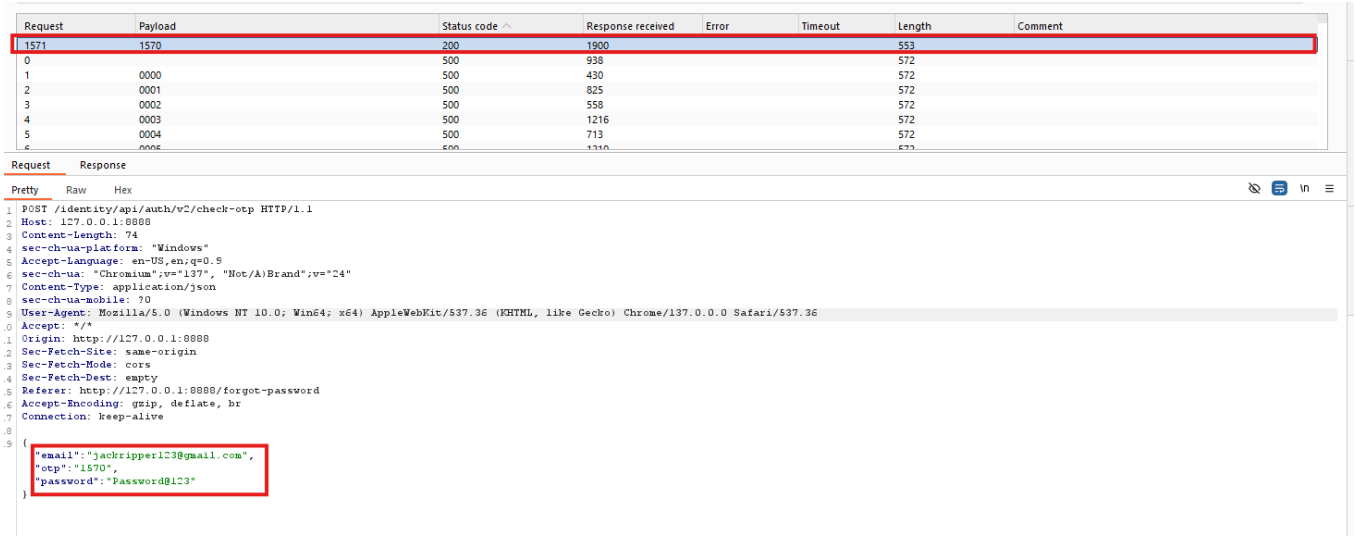

Figure 9.4 — v2 endpoint has no OTP limit; brute-force succeeds and password reset is confirmed

Confirmed Vulnerability

The deprecated v2 API endpoint imposed no OTP attempt limit. The brute-force attack successfully identified the correct OTP, enabling a password reset. This confirms that the v2 API version was never decommissioned, and its weaker security controls are actively exploitable.

Remediation

- Maintain a **comprehensive and up-to-date inventory** of all API endpoints, versions, environments, and associated services.
- Regularly identify and remove or securely disable **deprecated, unused, and legacy APIs** that are no longer required.
- Ensure that **development, testing, and staging environments** are properly secured and not unnecessarily exposed to the internet.
- Implement centralized API documentation and asset management processes to ensure all APIs are tracked and monitored throughout their lifecycle.
- Conduct periodic **API discovery, security assessments, and audits** to identify undocumented, shadow, or forgotten APIs and remediate associated risks.

API10 API10:2023
Unsafe Consumption of APIs

Overview

Unsafe Consumption of APIs is a vulnerability that occurs when an application blindly trusts and consumes data or services provided by external, third-party, partner, or internal APIs without adequately validating, sanitizing, or securing the interaction. Modern applications often rely on multiple APIs for functionality such as payment processing, authentication, data retrieval, and business operations. If the consuming application assumes that these APIs always return safe, accurate, and trustworthy data, attackers may exploit weaknesses in the API integration to compromise the application.

This vulnerability can arise when applications fail to validate responses from external APIs, improperly handle unexpected input, trust externally supplied data for security decisions, or use insecure communication channels. For example, a compromised or malicious API may return manipulated data that causes incorrect business decisions, inject malicious content into an application, expose sensitive information, or trigger unintended functionality. Additionally, weak authentication, insufficient authorization checks, or a lack of verification of third-party API responses can increase the risk of exploitation. Successful attacks may result in data breaches, privilege escalation, business logic manipulation, service disruption, or compromise of systems that depend on the untrusted API. Therefore, organizations should treat all external API data as untrusted input and apply appropriate validation, authentication, monitoring, and security controls before using it within their applications.

Remediation

- Treat all data received from external or third-party APIs as **untrusted input** and validate it before processing or storing it.
- Implement strong **authentication, authorization, and secure communication (HTTPS/TLS)** when interacting with external APIs.
- Verify and sanitize API responses to prevent the use of malicious, unexpected, or manipulated data in business processes.
- Apply appropriate **error handling, timeout settings, and response validation** to ensure the application behaves securely when external APIs fail or return unexpected results.
- Continuously monitor and assess third-party API integrations and conduct regular security reviews to identify and mitigate potential risks.

5. Summary of Findings

#	Vulnerability	Impact	Status
1	BOLA / IDOR	Unauthorized access to other users' vehicle locations	✓ Confirmed
2	Broken Authentication	Full account takeover via OTP brute-force	✓ Confirmed
3	BOPLA / Excessive Data Exposure	Sensitive user data exposed in API responses	✓ Confirmed
4	Unrestricted Resource Consumption	Layer 7 DoS via unbounded request repetition	✓ Confirmed
5	Broken Function Level Authorization	Admin-level deletion executed by standard user	✓ Confirmed
6	Business Flow Abuse	Account balance manipulation via negative quantities	✓ Confirmed
7	Server-Side Request Forgery	Arbitrary external URL fetched by server	✓ Confirmed
8	Security Misconfiguration	Insecure defaults and verbose error exposure	✓ Confirmed
9	Improper Inventory Management	OTP brute-force via deprecated API version	✓ Confirmed

Disclaimer

This lab was conducted entirely within the crAPI intentionally vulnerable application for authorized educational purposes only. All techniques documented herein must not be applied to any system without explicit written authorization. Unauthorized penetration testing is illegal.